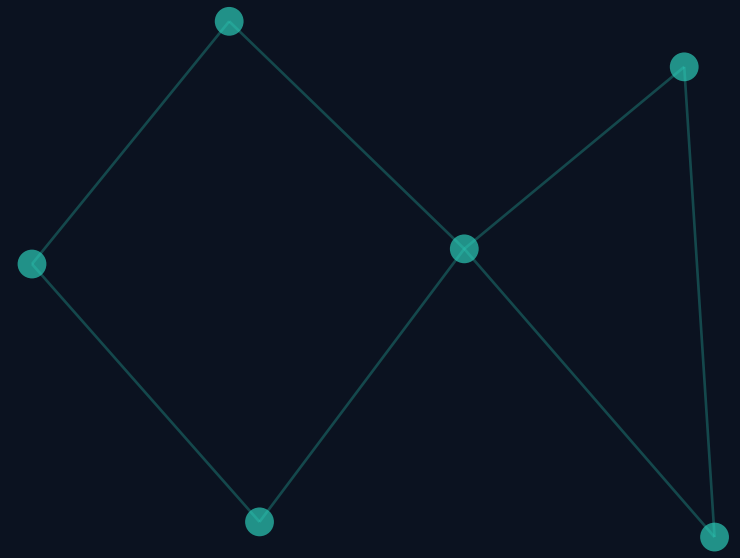
CLASE 9 · UNIDAD 4 · SEMANA 10

Memoria de largo plazo



Entre una sesión y la siguiente no sobrevive nada que no se haya escrito: qué se escribe, cómo se mantiene al día y cómo se vuelve a leer.

MEMORIA DE LARGO PLAZO · SECCIÓN 1 DE 6

Persistencia de la memoria entre sesiones

Entre una sesión y la siguiente no sobrevive nada que no se haya escrito: qué se guarda, dónde y cuándo se trae.

- **Persistencia de la memoria entre sesiones**
- Taxonomía y escritura de la memoria
- Mantenimiento de la memoria
- Recuperación de la memoria
- Evaluación de la memoria
- Modos de falla

Al día siguiente

Quedó abierta una deuda: la sesión se cerró. Todo lo que armamos en la primera mitad —el historial, el resumen, lo descargado— vivía adentro de una conversación.

Al día siguiente, el usuario vuelve. “¿Y al final, qué cambió para el bacalao?” El sistema no tiene idea de a qué se refiere “al final”. Y el usuario, que ayer contó que trabaja en una pesquera de Vigo, que sigue las cuotas del Báltico y que quiere las respuestas con el artículo citado, tiene que explicar todo de nuevo. El sistema le responde como a un desconocido.

Los productos de chat no hacen eso. ChatGPT guarda memorias del usuario desde 2024, y desde 2025 consulta además las conversaciones pasadas. Si el modelo no recuerda nada entre llamadas y la sesión se cerró, ¿de dónde sale eso? De lo que quedó escrito afuera. Como en Memento: sobrevive lo que se anotó.



cierre de sesión: sólo cruza lo que se escribió afuera

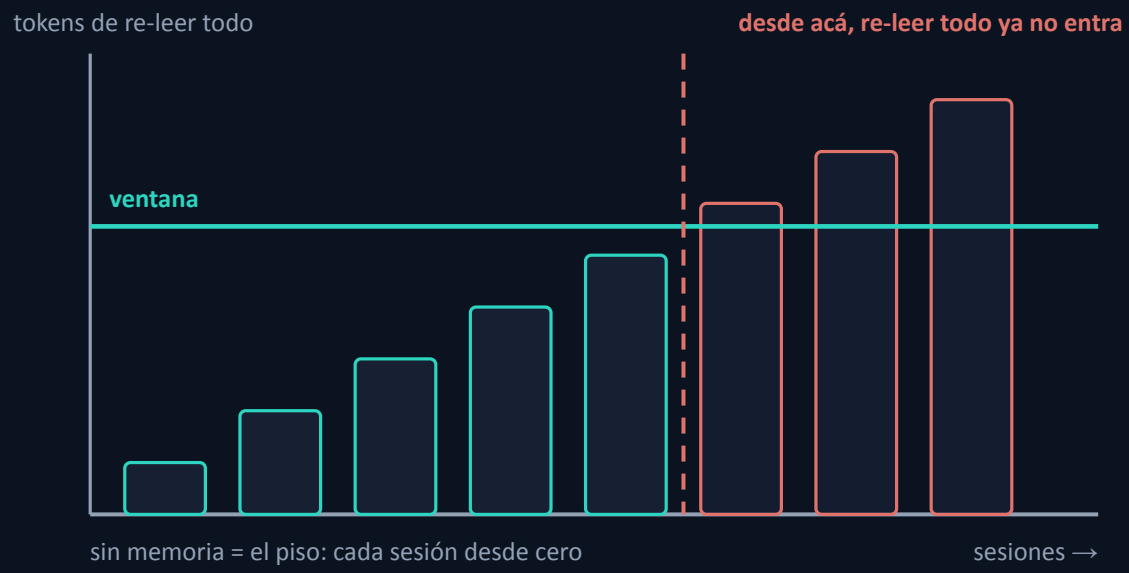
Entre una sesión y la siguiente no sobrevive nada que el sistema no haya escrito afuera.

Dos vallas: no recordar y re-leer todo

Antes de construir nada, las dos vallas. La primera es el piso: sin memoria, cada sesión desde cero. Todo tiene que ganarle a eso, y no es obvio: una memoria que trae algo equivocado puede quedar por debajo del piso.

La segunda es re-leer todo. Volver a mandar las sesiones pasadas enteras a la ventana: el "si entra, se pega", ahora entre sesiones. Funciona mientras entran. Después, el costo crece con la historia del usuario y la calidad cae con el largo. LongMemEval, un benchmark de memoria para asistentes, midió una caída de alrededor del 30% en la precisión de asistentes comerciales y modelos de contexto largo al recordar a lo largo de interacciones largas.

La regla que se sigue: declarar el régimen. Toda afirmación del tipo "esta memoria le gana al contexto completo" tiene que decir si el historial entraba en la ventana. Si entraba, se compara contra re-leer todo; si no, ni siquiera es una opción.



re-leer todo sirve mientras entra; después, ni siquiera es una opción

Una memoria de largo plazo tiene que ganarle a dos cosas: a no recordar nada, y a re-leer todo mientras entra. ▸ Notebook §9 — tres sesiones, y cuánto mide re-leerlas.

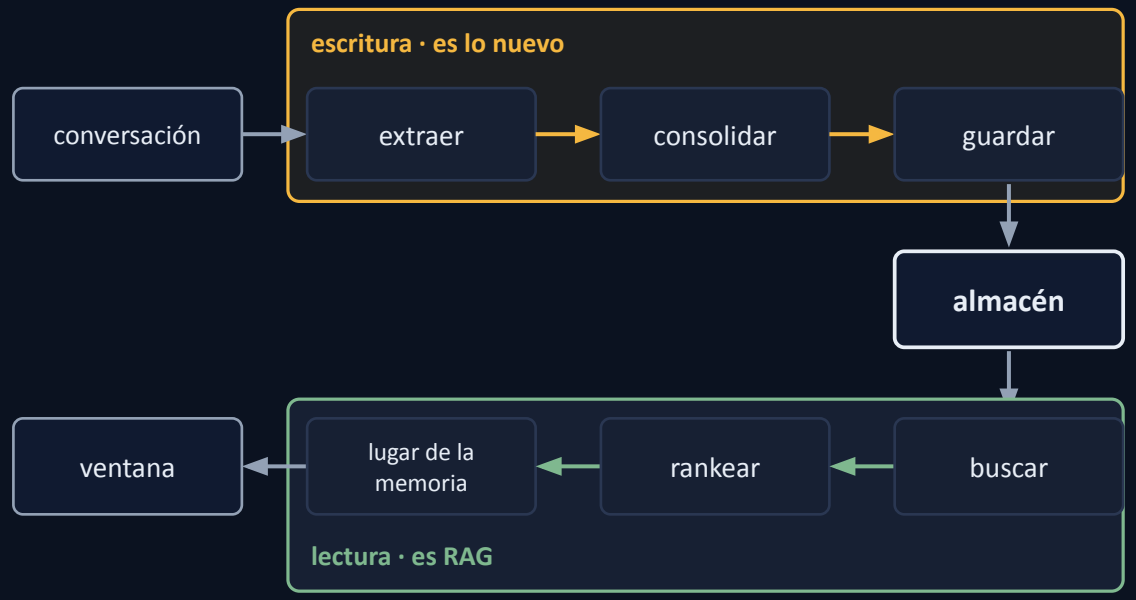
Una memoria es un corpus que escribe el sistema

Dos caminos.

1. El de escritura: de la conversación, decidir qué vale la pena guardar, y guardarlo afuera.
2. El de lectura: cuando llega una pregunta, buscar las memorias relevantes y ponerlas en el lugar de la memoria que la ventana tenía vacío.

Leer es RAG, y ya lo sabemos hacer. Un almacén de textos, un embedder, búsqueda por similitud, búsqueda híbrida, un re-puntuador: todo lo de la clase de RAG vale tal cual. Del lado de la lectura, la memoria de largo plazo es RAG sobre la propia historia del sistema.

Escribir es lo nuevo. En RAG, el corpus lo escribió otro y se limpió antes de empezar. Acá lo escribe el sistema, a mitad de la conversación, con el mismo modelo que habla. Puede estar mal, duplicado o viejo. Es la pregunta madre en su forma más pura: lo que se decide al escribir se decide sin saber qué se va a preguntar.



un solo almacén: se escribe sin conocer la pregunta, se lee cuando llegó

Leer es RAG, y ya lo sabemos hacer. Lo nuevo es escribir: el corpus lo escribe el sistema, mientras conversa, y se puede equivocar.

Dónde vive la memoria

No hay un almacén de memoria: hay varias formas de guardar. Y cada una responde bien un tipo distinto de pregunta.

Cuatro formas.

1. El mismo vector store de RAG: cada memoria es un texto con su vector, y se trae por similitud; sirve para “algo parecido a esto”.
2. Filas con campos y fechas, como en una base de datos: sirven para lo exacto y para el tiempo, “¿dónde trabaja ahora?”, “¿qué dijo en marzo?”.
3. Archivos: notas que una persona puede leer, corregir y versionar; sirven para la transparencia.
4. Grafos de entidades y relaciones: sirven para las conexiones, que son la última parte de la clase.

Los sistemas reales mezclan. La herramienta de memoria de la API de Anthropic guarda la memoria del modelo como una carpeta de archivos, del lado del cliente. Otros guardan un perfil en filas y los episodios en un vector store. La elección depende de qué preguntas se esperan.



cada forma de guardar, con la pregunta que responde bien

No hay un almacén de memoria: hay cuatro formas de guardar, y cada una responde bien un tipo distinto de pregunta.

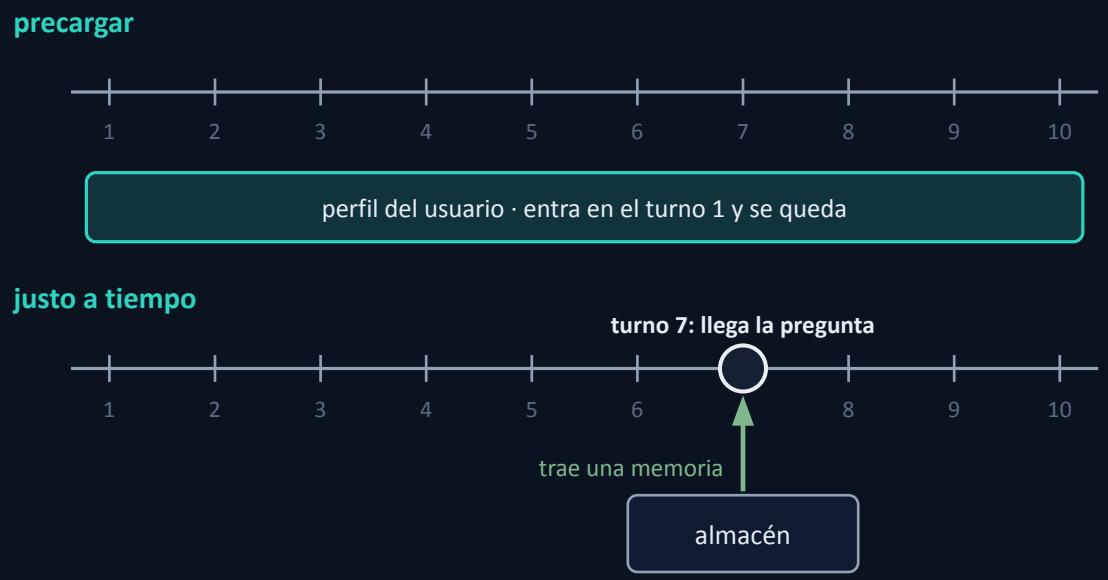
Traer la memoria justo a tiempo

Hay dos momentos para traer la memoria.

1. Al empezar la sesión: se carga en la ventana lo poco que siempre importa, como el perfil del usuario; es barato, predecible, y no depende de que una búsqueda acierte.
2. Justo a tiempo (just-in-time): cuando una pregunta lo necesita, se busca; escala a memorias enormes, pero cuesta una búsqueda por turno, y puede fallar.

La guía de context engineering de Anthropic describe el segundo. Los agentes guardan "lightweight identifiers" —rutas de archivos, consultas guardadas, enlaces— y con esas referencias cargan los datos en el contexto "at runtime". Y recomienda un híbrido: algo cargado al principio por velocidad, el resto explorado a demanda. Su ejemplo es un asistente de código: el archivo de instrucciones del proyecto se carga entero al arrancar, y los archivos del código se buscan a medida que hacen falta.

Es la pregunta madre otra vez. Precargar es decidir al escribir qué va a importar siempre; traer justo a tiempo es decidir al leer.



precargar decide al escribir; justo a tiempo, al leer

Lo poco que siempre importa se carga al empezar; lo mucho que importa a veces se busca cuando hace falta.

MEMORIA DE LARGO PLAZO · SECCIÓN 2 DE 6

Taxonomía y escritura de la memoria

Qué vale la pena guardar, en qué forma —evento, hecho o instrucción— y en qué momento escribirlo.

- Persistencia de la memoria entre sesiones
- **Taxonomía y escritura de la memoria**
- Mantenimiento de la memoria
- Recuperación de la memoria
- Evaluación de la memoria
- Modos de falla

QUÉ SE ESCRIBE · LA PREGUNTA

¿Qué vale la pena guardar?

El camino de escritura empieza con una pregunta. De todo lo que pasó en la sesión, ¿qué vale la pena guardar?

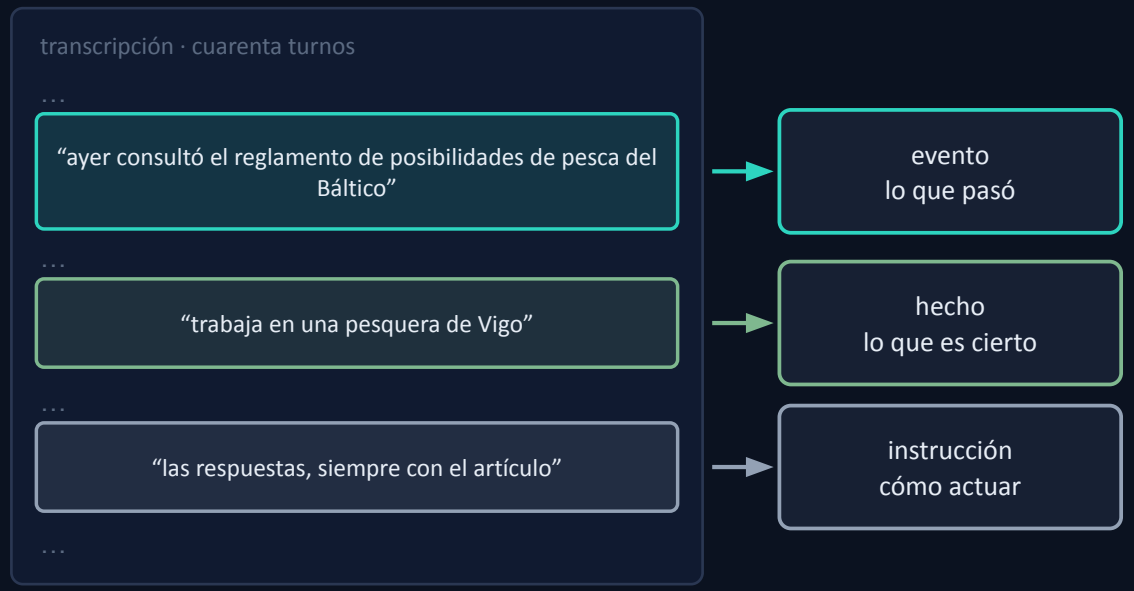
Las dos respuestas extremas fallan.

1. Guardar la transcripción entera es lo más simple, y no pierde nada; pero “¿en qué empresa trabajo?” queda enterrada en una sesión de cuarenta turnos, y hay que encontrarla ahí cada vez.
2. Guardar sólo los hechos extraídos es compacto, pero pierde lo que pasó: que ayer se consultó tal reglamento, en qué orden, para qué.

Porque una conversación mezcla tres cosas distintas.

1. Un evento: “ayer consultó el reglamento de posibilidades de pesca del Báltico”.
2. Un hecho: “trabaja en una pesquera de Vigo”.
3. Una instrucción: “las respuestas, siempre con el artículo”.

No se guardan, no se actualizan y no se traen igual.



tres cajones: no se guardan, no se actualizan y no se traen igual

Una conversación mezcla tres cosas: lo que pasó, lo que es cierto y cómo hay que actuar. Cada una se guarda distinto.

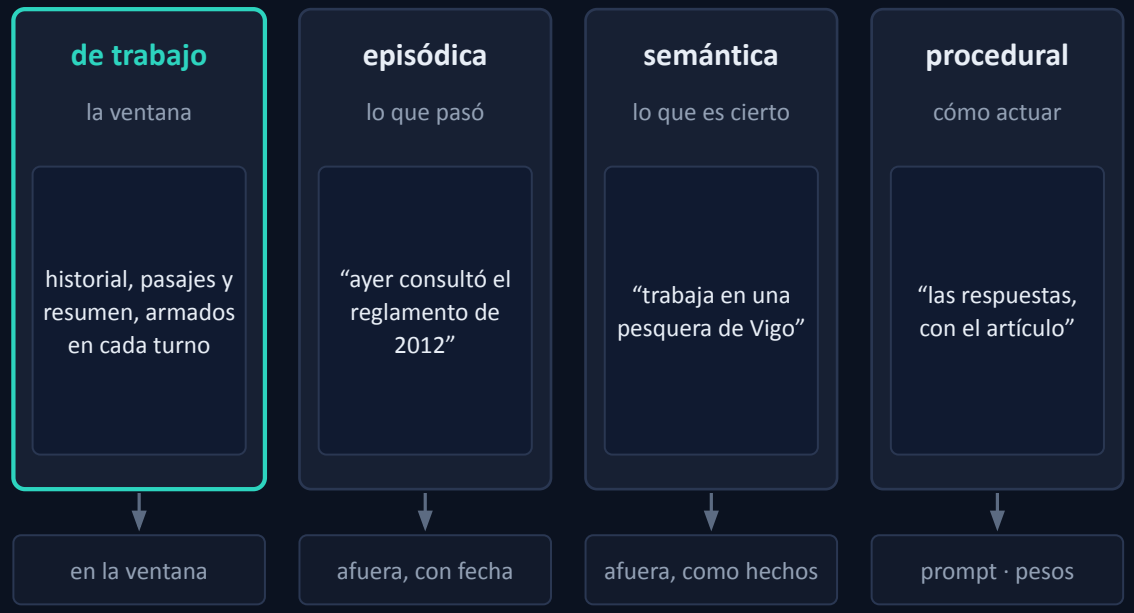
Cuatro memorias, con nombre

Las ciencias cognitivas ya tenían las palabras. Un marco de Princeton para agentes de lenguaje, CoALA, las tomó y las definió para estos sistemas. Son cuatro, y una ya la conocemos.

Las cuatro.

1. La memoria de trabajo: “active and readily available information… for the current decision cycle”; es la ventana que armamos en la primera mitad.
2. La episódica: “experience from earlier decision cycles”; lo que pasó.
3. La semántica: “knowledge about the world and itself”; lo que es cierto, incluido lo que se sabe del usuario.
4. La procedural, cómo actuar, que vive en dos lugares: “implicit knowledge stored in the LLM weights, and explicit knowledge written in the agent’s code”.

El vocabulario es estándar. Los docs de memoria de LangGraph usan la misma división —hechos, experiencias, instrucciones—, y conviene usarla para hablar de cualquier sistema: qué memoria tiene, dónde la guarda y quién la escribe.



abajo, dónde vive cada una · la de trabajo ya la conocemos

De trabajo es lo que se arma en la ventana; episódica, lo que pasó; semántica, lo que es cierto; procedural, cómo actuar.

Episódica: lo que pasó

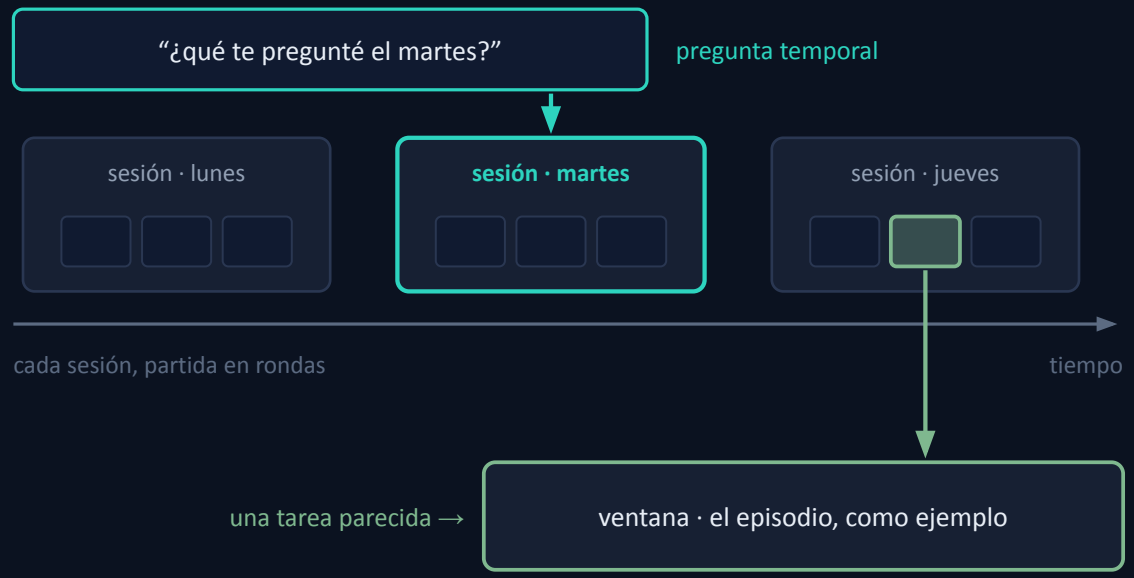
Registros de interacciones pasadas, con su fecha. Pueden ser sesiones enteras, turnos sueltos o los resúmenes de sesión de la primera mitad. Lo que los hace episódicos es que guardan un cuándo: no “el usuario sabe X”, sino “el martes se habló de X”.

La granularidad importa. LongMemEval descompone las sesiones en rondas antes de indexarlas, y le va mejor que guardando sesiones enteras: una sesión entera como un solo vector mezcla temas, el mismo problema de “un vector para los tres temas”.

Sirve para dos cosas.

1. La obvia: responder por el pasado, “¿qué te pregunté la semana pasada?”.
2. La menos obvia: mostrarle al modelo cómo resolvió algo parecido. Un episodio pasado que salió bien se trae como ejemplo, igual que los few-shot de prompt engineering, sólo que sacados de la propia historia del sistema.

Aprender de los episodios —reflexionar sobre lo que salió mal y corregir el comportamiento— es tema de planificación.



guarda un cuándo: no “sabe X”, sino “el martes se habló de X”

La memoria episódica guarda lo que pasó y cuándo: sirve para responder por el pasado y para mostrarle al modelo cómo resolvió algo parecido.

Semántica: lo que es cierto

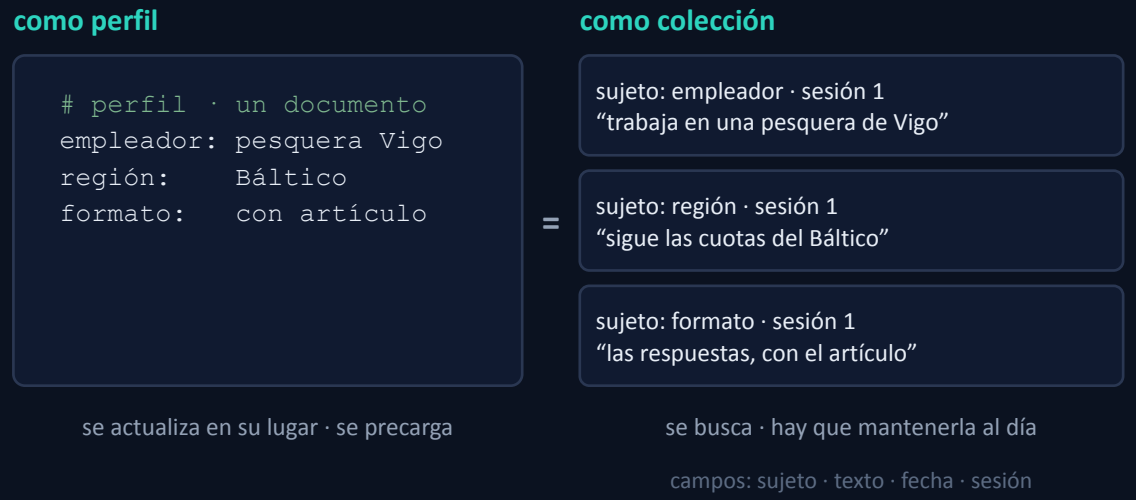
Hechos destilados de la conversación. Sobre todo, del usuario: dónde trabaja, qué sigue, qué prefiere. No es el registro de lo que pasó, sino lo que quedó establecido.

Dos formas de guardarla.

1. Un perfil: un documento por usuario, con campos, que se actualiza en su lugar y se carga entero al empezar la sesión, como vimos con precargar.
2. Una colección: muchos registros chicos, uno por hecho; escala mejor y se busca cuando hace falta, pero hay que mantenerla al día —de eso trata la próxima sección—.

Los docs de LangGraph nombran esta misma distinción.

La extracción es una llamada con un schema. Con structured outputs, el modelo lee la sesión y devuelve hechos candidatos con campos fijos. Esos campos son decisiones de diseño: qué cuenta como un hecho, de qué trata cada uno (el sujeto: empleador, región, formato), cuánto importa, de qué sesión salió.



los mismos tres hechos, guardados de dos formas

La memoria semántica destila hechos de la conversación. Como perfil se carga entera; como colección se busca, y hay que mantenerla al día. ▸ Notebook §10 — extraer los hechos de una sesión con un schema.

Procedural: cómo actuar

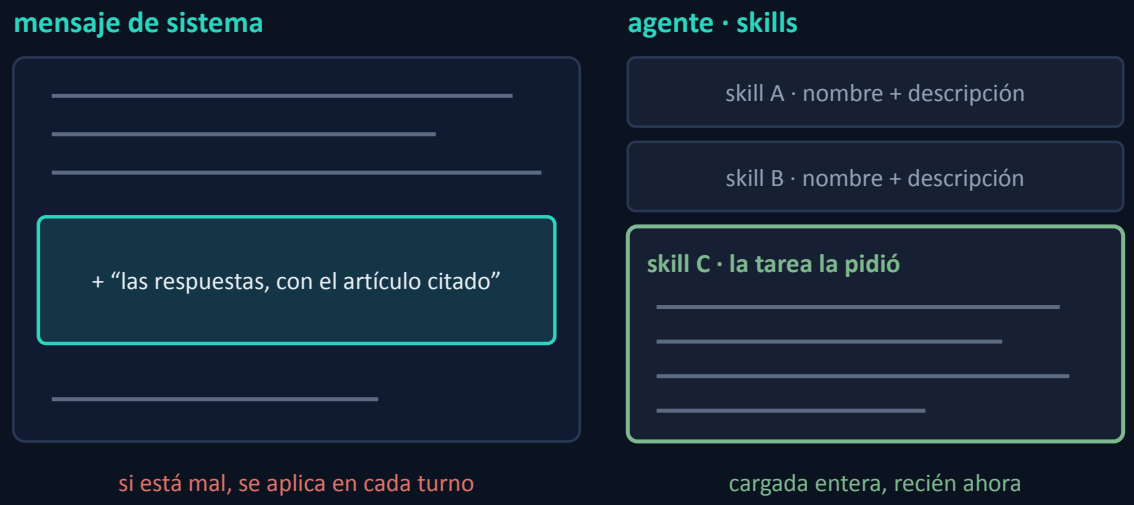
Cómo comportarse vive en dos lugares.

1. En los pesos (su entrenamiento).
2. En texto: el mensaje de sistema, las instrucciones, las preferencias de respuesta (“las respuestas, con el artículo citado”). Escribir memoria procedural es actualizar ese texto, a mano o con un proceso que lo optimiza.

Dos ejemplos reales.

1. Los asistentes de código cargan al arrancar un archivo de instrucciones del proyecto, que el equipo corrige con el uso.
2. Las skills: carpetas de instrucciones que un agente ve sólo por nombre y descripción, y carga enteras sólo cuando una tarea las pide; Anthropic lo llama “progressive disclosure”. Es carga justo a tiempo.

El riesgo, en palabras de CoALA. Escribir memoria procedural “is significantly riskier than writing to episodic or semantic memory, as it can easily introduce bugs or allow an agent to subvert its designers’ intentions”. Un hecho equivocado afecta las preguntas sobre él; una instrucción equivocada afecta todo.



progressive disclosure: el nombre siempre, el resto a demanda

La memoria procedural no guarda qué pasó ni qué es cierto: guarda cómo actuar. Y una instrucción equivocada se aplica en cada turno.

Guardar sólo lo que se quiere conservar

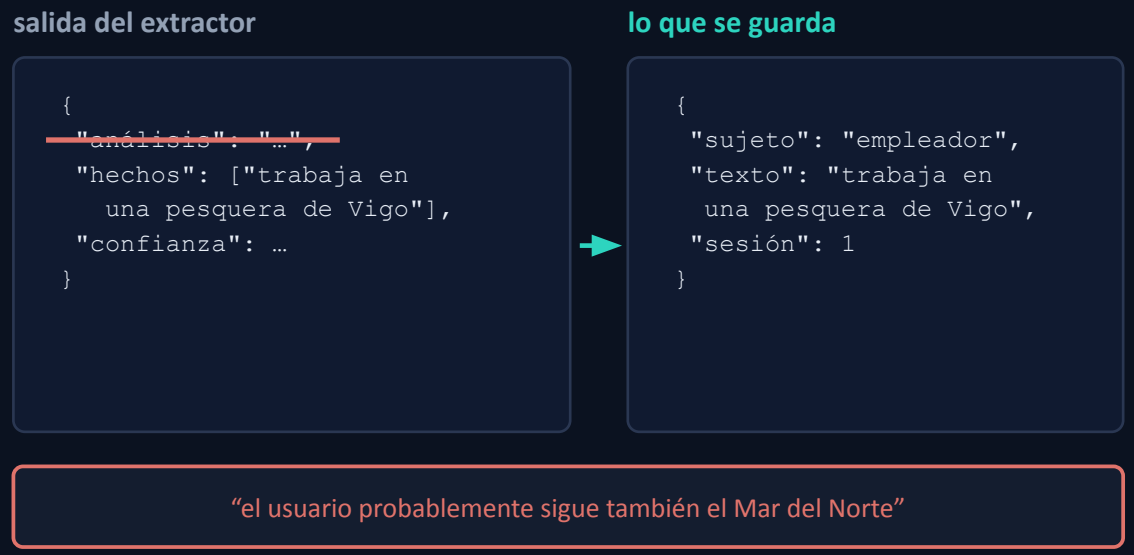
El extractor es un modelo, y escribe como un modelo. Puede guardar:

1. Una conjetura como si fuera un hecho (“el usuario probablemente sigue también el Mar del Norte”).
2. Su propio razonamiento.
3. Algo que nadie dijo, como los hechos inventados de la auditoría de resúmenes.

Y acá es peor que adentro de la sesión. Una vez guardado, se recupera en las sesiones que vienen como algo que dijo el usuario, y nadie lo vuelve a mirar. Lo que el historial hacía en una sesión, la memoria lo hace para siempre.

La regla.

1. Guardar sólo los campos que se quieren conservar, no la salida entera del extractor.
2. Verificar que ningún registro lleve razonamiento adentro.
3. Contar, auditando una muestra, cuántos registros están mal o nunca se dijeron: ese número es parte de evaluar una memoria.



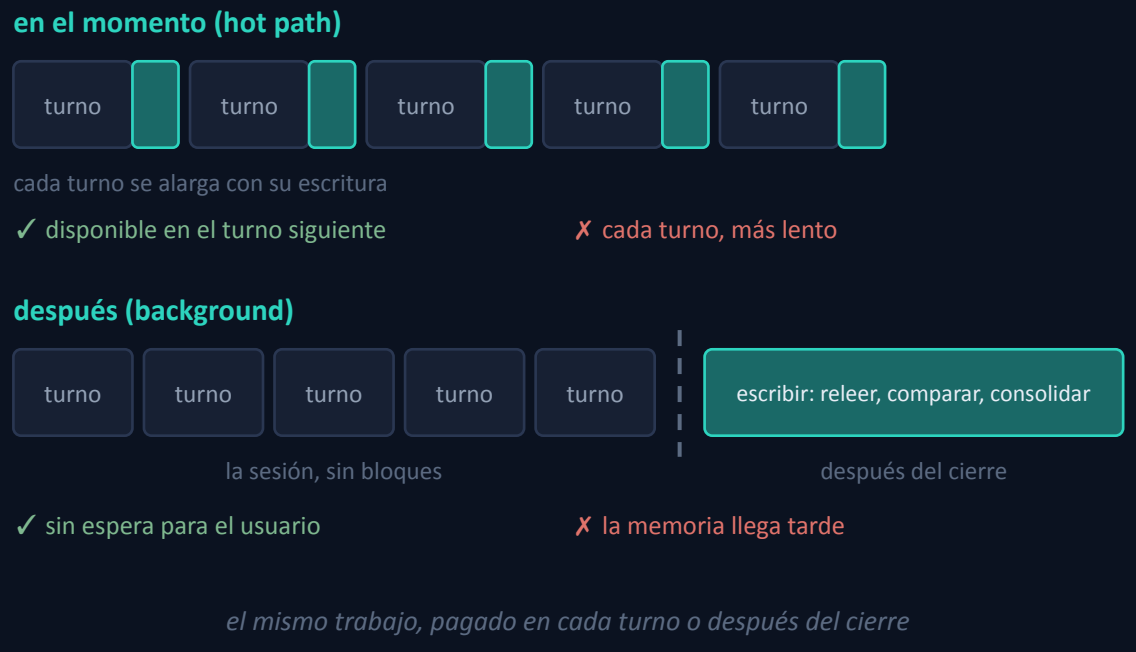
Lo que el extractor escribe, el sistema lo va a recordar como dicho. Una conjetura guardada es una memoria falsa que se lee para siempre.

Cuándo se escribe

Durante el turno, o después de la sesión. Las dos formas existen, y los docs de LangGraph las nombran así: escribir “in the hot path” o “in the background”.

En el momento (hot path). La memoria se escribe mientras la conversación pasa, y está disponible en el turno siguiente. Pero el turno del usuario se hace más lento, y el modelo decide qué guardar a mitad de la tarea, con poca perspectiva.

Después (background). Un proceso aparte lee la transcripción cuando la sesión terminó, o en un momento ocioso, y escribe con calma: puede releer, comparar, consolidar. No le agrega espera al usuario, pero la memoria llega tarde: lo dicho hoy no está hasta que el proceso corre. En la misma línea hay una idea más ambiciosa, el sleep-time compute: aprovechar el tiempo ocioso para procesar el contexto antes de que lleguen las preguntas.



Escribir en el momento hace la memoria inmediata y el turno más lento; escribirla después la hace cuidadosa y tardía.

MEMORIA DE LARGO PLAZO · SECCIÓN 3 DE 6

Mantenimiento de la memoria

Una memoria que sólo agrega se contradice: consolidar, invalidar con fecha y olvidar.

- Persistencia de la memoria entre sesiones
- Taxonomía y escritura de la memoria
- **Mantenimiento de la memoria**
- Recuperación de la memoria
- Evaluación de la memoria
- Modos de falla

El usuario cambió de trabajo

Escribir una vez es fácil. Lo difícil es que la memoria siga siendo cierta.

El usuario cambió de trabajo. En la sesión 1 dijo “trabajo en una pesquera de Vigo”; en la sesión 3, “me pasé a una de Gdansk”. Una memoria que sólo agrega guarda las dos. La búsqueda trae cualquiera, según cómo esté formulada la pregunta, y el modelo recibe una contradicción —dos hechos que chocan en la misma ventana, el context clash— o, peor, sólo el dato viejo.

Y se acumulan duplicados. “Trabaja en Vigo”, dicho de cinco formas en cinco sesiones, son cinco registros casi iguales que se reparten el top-k y desplazan todo lo demás: el consenso falso que vimos con la fusión de rankings en RAG, ahora adentro de la memoria. LongMemEval trata la actualización de conocimiento como una habilidad aparte, porque es de las que más fallan.



sólo agregar: el top-k se llena de lo viejo y de lo repetido

Una memoria que sólo agrega se contradice y se repite. Escribir no es agregar: es mantener.

Agregar, actualizar, borrar o nada

Antes de guardar, comparar con lo que ya hay. El camino de escritura de Mem0, un sistema de memoria de producción, lo hace así: por cada hecho candidato, primero recupera las memorias existentes más parecidas, y después un modelo elige una de cuatro operaciones.

1. ADD: es nuevo, se agrega.
2. UPDATE: corrige o completa uno que ya está.
3. DELETE: el nuevo lo contradice, y el viejo se borra.
4. NOOP: ya se sabía, no se hace nada.

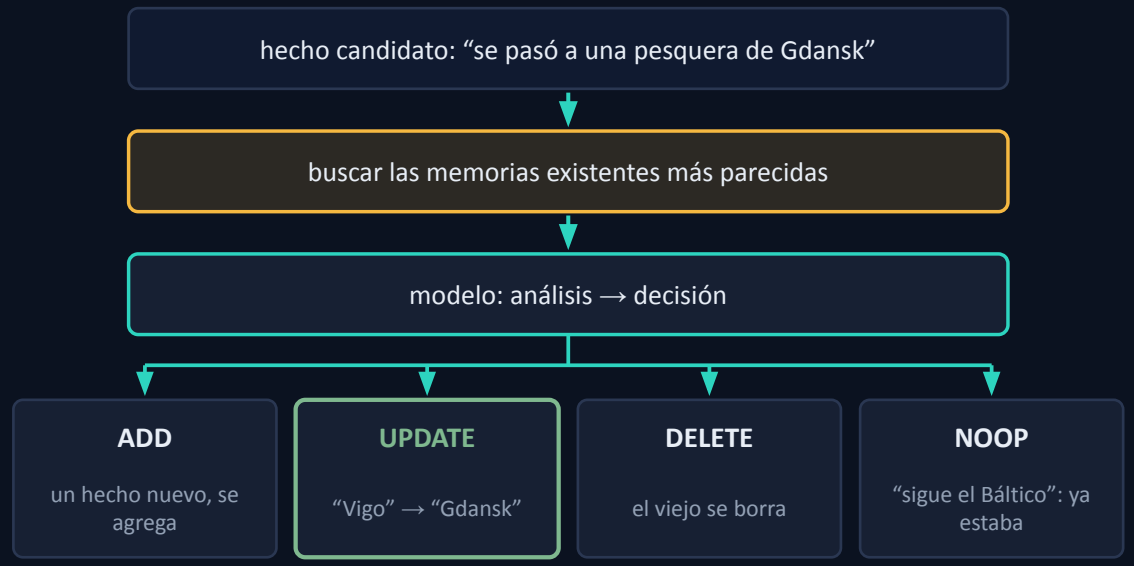
Es la consolidación (consolidation) de la memoria.

Es un structured output, con su orden. La decisión sale de un schema: primero un campo de análisis, después la operación, después a qué registro afecta. El razonamiento va antes que la decisión, por lo que vimos en prompt engineering sobre el orden de los campos.

Y tiene dos puntos donde fallar. Es una recuperación seguida de una decisión:

1. Si la búsqueda no trae el registro viejo, se agrega un duplicado.
2. Si el modelo decide mal, se pisa lo que no había que pisar.

En el notebook, el modelo del curso decidió mal el cambio de trabajo en las tres corridas, y distinto cada vez. A cambio, es el antídoto del colapso de contexto: se toca un registro, no se reescribe un resumen.



en el notebook, tres corridas: ADD · DELETE · ADD — nunca UPDATE

y “sigue el Báltico”, que era NOOP, terminó borrado una vez y duplicado dos

Cada hecho nuevo se compara con lo que ya hay antes de guardarse: agregar, actualizar, borrar o no hacer nada. ▸ Notebook §11 — las cuatro operaciones sobre el cambio de trabajo.

Invalidar, no borrar

Borrar destruye la historia. Si “trabaja en Vigo” se borra cuando llega “trabaja en Gdansk”, la memoria responde bien “¿dónde trabajo?”, pero ya no puede responder “¿dónde trabajaba antes?” ni “¿qué te dije en marzo?”.

Dos relojes por hecho. Zep, otro sistema de memoria, guarda cada hecho con dos líneas de tiempo: cuándo fue cierto en el mundo, desde y hasta cuándo valió, y cuándo lo registró el sistema, que sirve para auditar. Ante una contradicción no borra: le pone fecha de fin al hecho viejo —“invalidates the affected edges by setting their t_invalid”— y el nuevo arranca ahí. A este diseño se le dice bitemporal, y borrar se reemplaza por invalidar.

No hace falta un grafo para esto. Zep guarda los hechos en un grafo, pero la idea entra en dos columnas de fecha en una fila: válido desde, válido hasta. Y habilita algo que la memoria necesitaba: el razonamiento temporal, otra de las habilidades que mide LongMemEval. Con un límite: la fecha filtra qué era cierto, no qué es relevante.



nota de laboratorio · “¿y en mayo?” trajo “se interesa por el Mar del Norte”:
la fecha dejó pasar a Vigo, pero la similitud prefirió otro registro vigente

Lo que dejó de ser cierto no se borra: se le pone fecha de fin. Así la memoria responde qué es cierto hoy y qué era cierto antes. ▸ Notebook §11 — invalidar con dos fechas, y preguntar por el pasado.

Olvidar, también a largo plazo

Una memoria que nunca olvida se llena de lo que ya no importa. Cada consulta pasajera, cada detalle de una tarea que terminó hace meses, compite en el top-k con lo que sí importa.

Olvidar como las personas. MemoryBank, un trabajo sobre memoria de largo plazo para modelos de lenguaje, toma la curva del olvido de Ebbinghaus: las memorias se debilitan con el tiempo y se refuerzan cada vez que se recuerdan, según el tiempo transcurrido y su importancia. Otros sistemas usan algo más simple:

1. Fechas de vencimiento.
2. Umbrales de importancia bajo los cuales un registro se archiva.

Y un caso que exige borrar de verdad. Cuando el usuario pide que algo se olvide, invalidar no alcanza: el dato tiene que desaparecer. Lo retomamos con la privacidad. El olvido selectivo, además, es difícil: un benchmark reciente de memoria para agentes lo mide como una de sus cuatro competencias, y ningún sistema evaluado domina las cuatro.



decae con el tiempo, se refuerza al recordarse; bajo el umbral, se archiva

Una memoria que nunca olvida se llena de lo que ya no importa. Decaer, reforzar y borrar a pedido también son operaciones de memoria.

MEMORIA DE LARGO PLAZO · SECCIÓN 4 DE 6

Recuperación de la memoria

Parecido no alcanza: traer memorias por relevancia, recencia e importancia, dentro del presupuesto de la ventana.

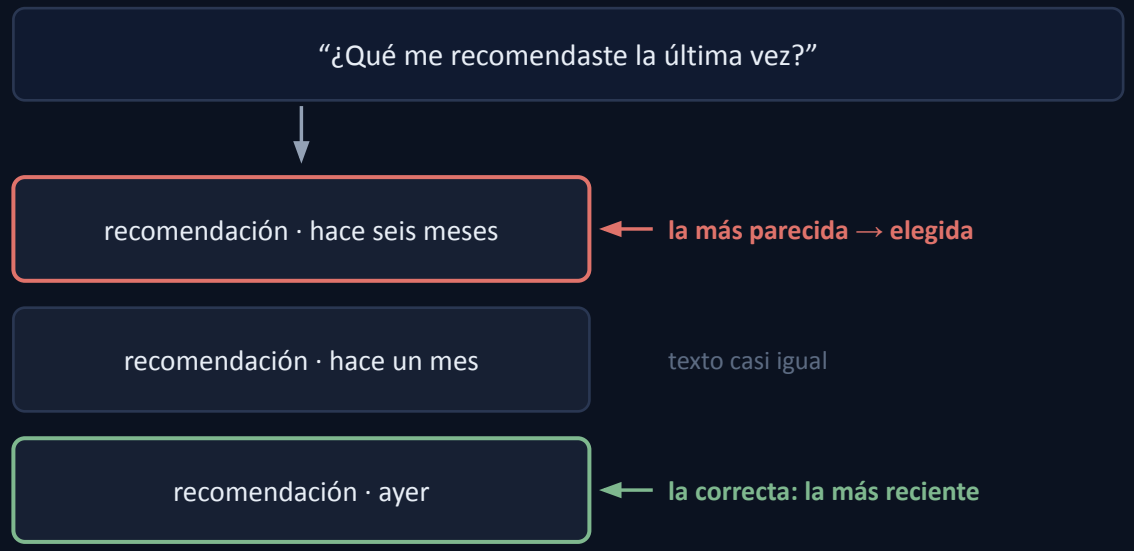
- Persistencia de la memoria entre sesiones
- Taxonomía y escritura de la memoria
- Mantenimiento de la memoria
- **Recuperación de la memoria**
- Evaluación de la memoria
- Modos de falla

Parecido no alcanza

Ya hay memorias guardadas y al día. Falta traerlas cuando hacen falta, y acá el RAG de siempre se queda corto.

La similitud no sabe de tiempo ni de peso. "¿Qué me recomendaste la última vez?" trae la memoria más parecida a la pregunta, que puede ser de hace seis meses, no la más reciente. "¿Qué es lo más importante que te pedí?" ordena por palabras, no por importancia. Y la pregunta rara vez repite las palabras de la memoria: es el caso de NoLiMa, la búsqueda sin coincidencia literal.

La memoria necesita dos dimensiones que RAG no usaba: cuándo pasó y cuánto importa. En un corpus de reglamentos, todos los pasajes tenían, de entrada, la misma vigencia y el mismo peso; en la memoria de una persona, no.



Para la memoria, parecido no alcanza: también importa cuándo pasó y cuánto importa.

Relevancia, recencia, importancia

Un trabajo de Stanford lo resolvió con una suma. En Generative Agents cada memoria se trae por un puntaje de tres términos:

La relevancia: el coseno entre la pregunta y la memoria, el de embeddings.

La recencia: un decaimiento exponencial de 0,995 por hora “since the memory was last retrieved”, así que recordar una memoria la refresca.

La importancia: un número de 1 a 10 que el modelo le pone al escribirla, de “brushing teeth” (1) a “a break up” (10).

Los tres se normalizan y se suman. Cada término se lleva a [0, 1] con min-max, y se suman con pesos iguales. Las memorias de puntaje más alto entran a la ventana.

Indexar mejor también ayuda. LongMemEval suma dos ideas en la misma dirección: indexar cada memoria también por los hechos que se extrajeron de ella, y agregarle a la consulta el rango de tiempo que implica (“la semana pasada” → fechas).



la más parecida (m1) queda afuera: la suma reordena

Una memoria se trae por una suma: cuánto se parece a la pregunta, qué tan reciente es y cuánto importa. ▸ Notebook §12 — los tres puntajes, con sus rangos impresos.

La trampa de normalizar

Min-max estira cualquier rango a [0, 1]. Si en un conjunto de memorias la recencia va de 0,2 a 0,9, normalizar la lleva a 0–1 y conserva el orden: bien. Pero si casi no varía, también la estira a 0–1.

El caso. En una prueba guionada, todas las memorias se escriben con segundos de diferencia, y la recencia queda entre 0,99981 y 0,99987. Normalizada, ese ruido pasa a ocupar el rango completo y pesa tanto como la relevancia: un tercio del puntaje lo decide la milésima de segundo en que se escribió cada memoria. Lo mismo pasa si el modelo le puso 5 de importancia a casi todo.

La defensa. Usar las fechas del escenario, no las del reloj de la computadora, e imprimir el rango de cada término antes de sumar. Es la misma trampa que vimos en RAG: los puntajes de dos canales no se suman sin mirar sus escalas. La fusión por rankings la esquivó usando puestos en vez de puntajes.



un tercio del puntaje, decidido por segundos de diferencia

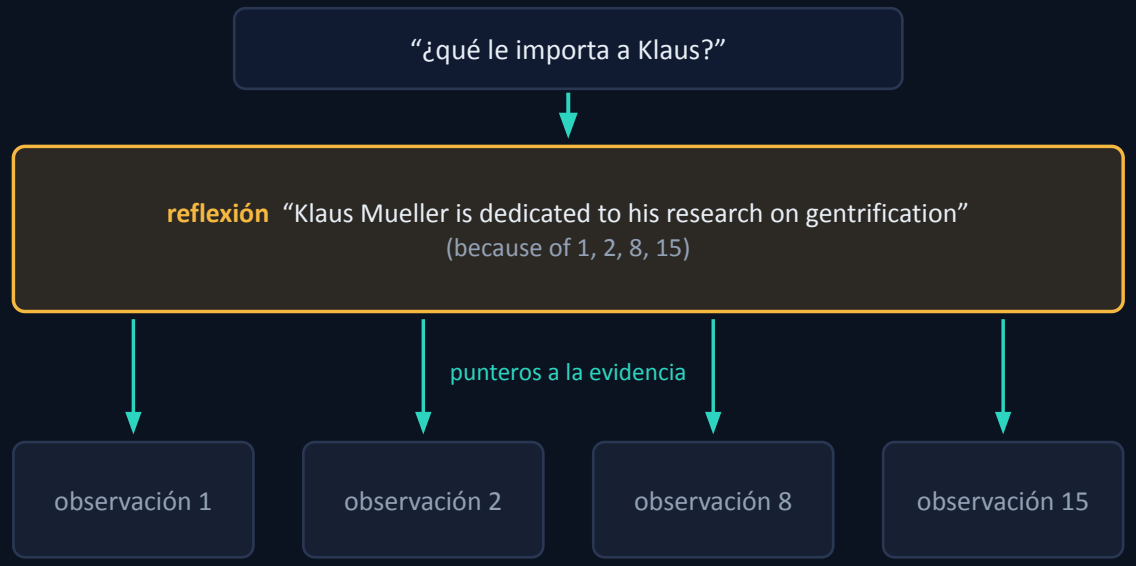
Normalizar estira cualquier rango a [0, 1], también el que es puro ruido. Antes de sumar, mirar cuánto varía cada término.

Reflexión: recuerdos de recuerdos

Las observaciones sueltas no dicen lo que significan. Cien memorias del tipo “leyó un libro sobre gentrificación”, “escribió sobre gentrificación” no responden “¿qué le importa?”. Generative Agents agrega un paso: reflexionar.

Cuándo y cómo. Cuando la importancia de los eventos recientes suma más que un umbral —150 en el paper—, el agente se hace preguntas sobre sus memorias recientes y escribe conclusiones de nivel más alto de vuelta en la memoria, con punteros a la evidencia: “Klaus Mueller is dedicated to his research on gentrification (because of 1, 2, 8, 15)”. Las reflexiones se traen como cualquier memoria, y se puede reflexionar sobre reflexiones: es el árbol de resúmenes, aplicado a la memoria.

La frontera. Acá la reflexión sirve para recordar mejor. Usarla para actuar mejor —mirar lo que salió mal en un episodio y corregir el comportamiento— es tema de planificación.



la pregunta trae la reflexión; la reflexión apunta a sus observaciones

Reflexionar es resumir la memoria en ideas de nivel más alto, que después se traen como cualquier recuerdo, con punteros a su evidencia.

La memoria entra por el presupuesto

El lugar vacío de la ventana se llena. Las memorias que se traen van al segmento de memoria del ensamblador, el que estaba vacío desde la primera mitad. Ahora compiten tres inquilinos variables por el mismo presupuesto: pasajes, historial y memoria.

La memoria no es contexto gratis. Una memoria equivocada desvía la respuesta tanto como un pasaje equivocado, y con más autoridad: parece algo que dijo el usuario. Y demasiadas memorias distraen igual que demasiados pasajes: la ventana efectiva no cambió.

Vuelve el criterio de la primera mitad.

1. Contar: cuántas memorias entran.
2. Presupuestar: a costa de qué.
3. Ordenar: en qué posición.

Y la vara de siempre: ¿esta memoria le gana a re-leer las sesiones pasadas, si entraran?



mismo total: lo que entra de memoria sale de historial y pasajes

Las memorias compiten por la misma ventana que los pasajes y el historial: entran por el presupuesto, no por encima de él.

La memoria como filtro de búsqueda

Lo que el sistema sabe del usuario puede afinar la búsqueda. No sólo se trae a la ventana: también puede acotar la búsqueda en el corpus. “Sigue el Báltico” se vuelve un filtro de región sobre los metadatos, los que vimos en RAG. “Al día de hoy” se vuelve un filtro de vigencia.

Y un filtro de más esconde la respuesta correcta. El usuario pregunta por una cuota del Mar del Norte. Con el filtro del Báltico, el acto relevante queda afuera, y el modelo, que no lo ve, no puede decir lo que tendría que decir: que ese reglamento no es de su región. Responde con lo que encontró adentro del filtro, o se abstiene sin saber por qué.

El filtro también se mide. En las preguntas que lo necesitan, ¿mejoró la recuperación? Y en las que no, ¿sacó de la vista el pasaje que permitía abstenerse bien?



el filtro saca de la vista el pasaje que permitía abstenerse bien

Lo que el sistema sabe del usuario puede afinar la búsqueda. Y un filtro de más esconde justo el pasaje que permitía decir “eso no aplica”.

MEMORIA DE LARGO PLAZO · SECCIÓN 5 DE 6

Evaluación de la memoria

Una memoria tiene más superficies de falla que un RAG: se mide por habilidad y contra dos vallas, no por un promedio.

- Persistencia de la memoria entre sesiones
- Taxonomía y escritura de la memoria
- Mantenimiento de la memoria
- Recuperación de la memoria
- **Evaluación de la memoria**
- Modos de falla

¿Cómo se sabe si una memoria sirve?

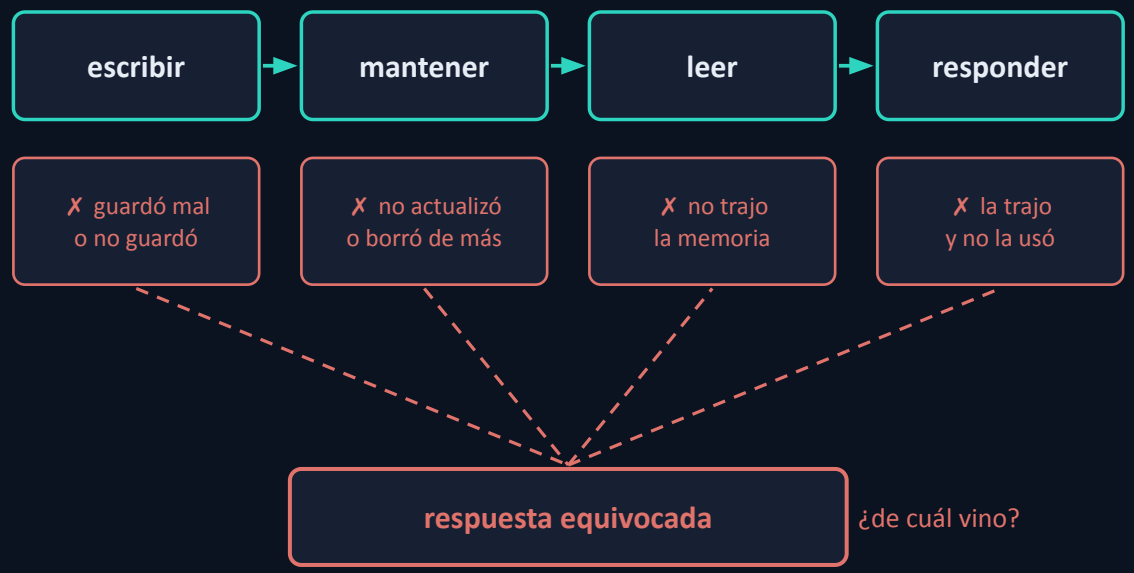
Armamos una memoria entera. Falta lo que la clase de RAG enseñó a no saltear: medirla.

Un número al final esconde dónde falló. RAG tenía dos superficies de falla: recuperar y generar. La memoria tiene cuatro:

1. Escribir (el extractor guardó algo mal, o no guardó).
2. Mantener (no actualizó, o borró lo que no debía).
3. Leer (no trajo la memoria que hacía falta).
4. Responder (la trajo, y el modelo no la usó).

Una respuesta equivocada puede venir de cualquiera de las cuatro.

Y las dos vallas también se miden. Sin memoria y re-leer todo, sobre las mismas preguntas. Sin esos dos números al lado, el de la memoria no dice nada.



una respuesta equivocada, cuatro orígenes posibles

Una memoria tiene más superficies de falla que un RAG: puede escribir mal, no actualizar, no traer o no usar. Un número al final no dice cuál.

Cinco preguntas distintas

Un benchmark que mide por habilidad. LongMemEval (ICLR 2025) arma 500 preguntas sobre historiales de chat y las separa en cinco habilidades; cada una pone a prueba una parte distinta del pipeline.

Las cinco.

1. Extracción de información: recordar un detalle dicho una vez.
2. Razonamiento multi-sesión: combinar hechos de sesiones distintas.
3. Razonamiento temporal: cuándo, antes o después.
4. Actualización de conocimiento: dar el valor más reciente después de un cambio, como el trabajo en Gdansk.
5. Abstención: decir “no lo sé” cuando algo nunca se dijo.

Además, las sesiones que contienen cada respuesta están marcadas: la recuperación se puntúa sin juez.

En nuestro caso. Dos preguntas por habilidad, sobre tres sesiones guionadas con hechos plantados, reportadas como conteos —1 de 2, no 50%— porque son pocas. Es la vara de la clase; la versión con muchas preguntas es parte del segundo proyecto.

“¿cómo se llama nuestro arrastrero?”	**extracción** prueba: escribir
“¿sobre qué mares te consulté?”	**multi-sesión** prueba: leer
“¿dónde trabajaba en mayo?”	**temporal** prueba: leer
“¿dónde trabajo?”	**actualización** prueba: mantener
“¿qué cuota de atún te pedí?”	**abstención · nunca se dijo** prueba: responder

dos preguntas por habilidad · conteos, no porcentajes

Una memoria se mide por habilidades, no por un promedio: extraer, combinar, ubicar en el tiempo, actualizar y saber que no sabe. ▸ Notebook §13 — la vara de la clase, dos preguntas por habilidad, en conteos.

MEDIR · SIN JUEZ · ANTES DEL JUEZ

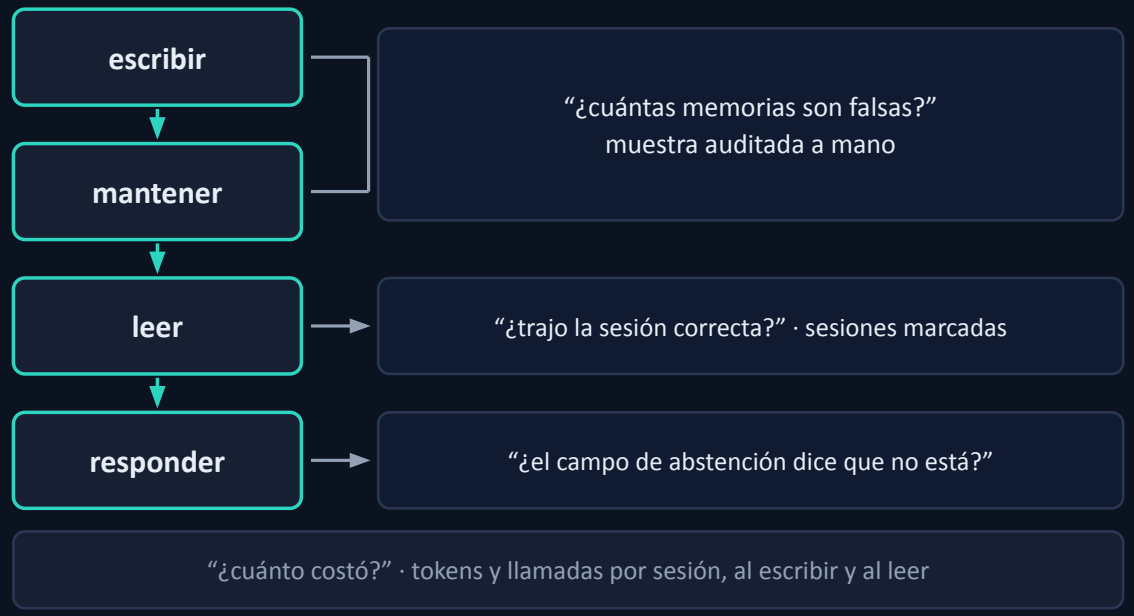
Lo que se puede contar

Como en RAG: primero la vara que no necesita juez. Un juez LLM sirve para las respuestas abiertas, y trae sus sesgos. Antes de llamarlo, hay medidas que se pueden contar directamente.

Cuatro, una por superficie.

1. Para leer: si la búsqueda trajo la sesión donde se dijo el hecho, porque esas sesiones están marcadas.
2. Para escribir: cuántas memorias guardadas están mal o nunca se dijeron, auditando una muestra a mano.
3. Para el costo: tokens y llamadas al modelo por sesión, de escritura y de lectura.
4. Para abstenerse: un campo del schema de la respuesta ("¿la respuesta está en la memoria?"), no una frase que haya que buscar en el texto.

Lo que el juez agrega, y lo que no. El juez sirve para decidir si una respuesta abierta es correcta. No reemplaza a ninguna de estas cuatro medidas: si una falla, ya se sabe dónde.



el juez, sólo para las respuestas abiertas

Antes del juez, lo que se puede contar: si trajo la sesión correcta, cuántas memorias guardadas son falsas, y cuánto costó.

Un benchmark que no habla castellano

LoCoMo es el otro benchmark de referencia. Conversaciones muy largas, de unas diecinueve sesiones, con preguntas de un salto, de varios saltos, temporales, de conocimiento general, y unas adversariales: preguntas pensadas para que el agente reconozca que no tienen respuesta.

La letra chica. En el código oficial del benchmark, una pregunta adversarial vale 1 si la respuesta contiene 'no information available' o 'not mentioned', y 0 si no. Una abstención correcta en castellano —“no se menciona”— vale cero. Y cualquier respuesta que diga “not mentioned” por error, vale uno.

La lección es general. Un benchmark mide lo que su puntuación cuenta, y antes de creerle un número hay que leer cómo se calcula, como con la letra chica de RAGAS. Para nuestro caso: la abstención se puntúa desde un campo del schema, no buscando una frase en el texto.

```
# task_eval/evaluation.py · adversariales
if 'no information available' in output.lower() \
   or 'not mentioned' in output.lower()
```

dos abstenciones correctas

“Not mentioned in the conversation.”	→ 1
“No se menciona en la conversación.”	→ 0

la puntuación cuenta una frase en inglés, no la abstención

Un benchmark mide lo que su puntuación cuenta. Si cuenta una frase en inglés, una abstención correcta en castellano vale cero.

¿Gana la memoria o el contexto completo?

Los proveedores de memoria discuten en público.

1. El paper de Mem0 presenta su memoria cerca del contexto completo a una fracción del costo.
2. Zep, un competidor, publicó una réplica: las conversaciones de LoCoMo entran enteras en una ventana moderna, así que re-leer todo es la comparación honesta, y gana.
3. Letta, un tercero, mostró un agente que guardaba la memoria en archivos planos y sacaba 74% en el mismo benchmark: “memory is more about how agents manage context than the exact retrieval mechanism used”.

Dos lecciones para leer estos números.

1. Los proveedores miden sus propios productos, y eligen los benchmarks.
2. Un benchmark cuyo historial entra entero en la ventana no prueba memoria de largo plazo: prueba si un sistema le gana a pegar todo, en un régimen donde pegar todo es posible.

Por eso se declara el régimen. La pregunta útil no es qué memoria gana, sino a partir de qué largo de historial deja de convenir re-leer todo, para este caso y este presupuesto.

sistema	resultado	fuente	¿entraba el historial?
contexto completo	re-leer todo gana	Zep · réplica	**SÍ**
Mem0	cerca del completo, a una fracción del costo	Mem0 · paper	**SÍ**
agente con archivos	74,0% en LoCoMo	Letta	**SÍ**

las tres fuentes son proveedores

las tres corren en el régimen donde pegar todo es posible

La disputa sobre qué memoria gana se resuelve declarando el régimen: si el historial entra, re-leerlo es difícil de superar.

MEMORIA DE LARGO PLAZO · SECCIÓN 6 DE 6

Modos de falla

Un error escrito se lee en cada sesión que viene: persistencia, sesgo, inyección y privacidad.

- Persistencia de la memoria entre sesiones
- Taxonomía y escritura de la memoria
- Mantenimiento de la memoria
- Recuperación de la memoria
- Evaluación de la memoria
- **Modos de falla**

DÓNDE FALLA · LA FALLA QUE PERSISTE

Un error que se recuerda para siempre

Una memoria que funciona también puede fallar. Y de formas que una sesión no podía, porque una sesión termina.

En una sesión, un error muere al cerrarla. Un resumen que inventó algo contamina esa conversación, y nada más. En la memoria de largo plazo, un registro equivocado se escribe una vez y se lee en cada sesión que viene, con la autoridad de algo que "el usuario dijo". Nadie lo vuelve a mirar, porque nadie sabe que está ahí.

Como en Memento. El protagonista confía en sus notas porque las escribió él. No las puede verificar: no recuerda haberlas escrito. Y una de ellas es una mentira que se escribió a sí mismo, sabiendo que la iba a creer. Un sistema que toma su propia memoria como verdad tiene exactamente esa estructura: lo que escribió es lo que es cierto.



En una sesión, un error muere al cerrarla. En la memoria, se escribe una vez y se lee para siempre, con la autoridad de algo que "el usuario dijo". ▸ Notebook §14 — un registro que nadie dijo, leído en dos preguntas.

Recuerdos fuera de lugar, sesgos más firmes

A veces la memoria acierta y el problema es usarla. PersistBench, presentado en ICML 2026, midió en 18 modelos —comerciales y abiertos— dos formas en que las memorias de largo plazo hacen daño aunque sean ciertas.

Las dos.

1. La filtración: el modelo mete memorias en conversaciones donde no tienen nada que hacer —el empleador del usuario apareciendo en una pregunta sobre otra cosa—, con una tasa mediana de falla del 53%.
2. El refuerzo de sesgos: las memorias guardadas “insidiously reinforce user biases”, con una tasa mediana de falla del 97%. Si el usuario dejó una opinión, el modelo empieza a darle la razón cada vez más.

Es la complacencia del alineamiento, amplificada. Cuando vimos alineamiento, el modelo tendía a decirle al usuario lo que quería escuchar dentro de una conversación. Con memoria, esa tendencia tiene de dónde agarrarse en todas las conversaciones que vienen.



memorias ciertas, usadas donde no corresponde

La memoria no sólo recuerda mal: recuerda donde no corresponde, y le devuelve al usuario sus propios sesgos, cada vez más firmes.

Una inyección que se queda

RAG cerró con una frontera. Todo lo que entra al índice llega al prompt; un pasaje puede traer instrucciones dirigidas al modelo, y el sistema no las distingue del contenido. Con memoria, esa frontera empeora.

Si una instrucción queda escrita en la memoria, vuelve en cada sesión. Puede entrar por:

1. Un pasaje recuperado.
2. Un documento compartido.
3. Un turno malicioso.

El extractor la guarda como un hecho o una preferencia. AgentPoison (2024) lo midió: envenenando menos del 0,1% de la memoria de un agente logró más de 80% de éxito, con menos de 1% de impacto visible. Y la memoria procedural es el peor lugar: una instrucción falsa ahí se aplica en todo, como advertía CoALA.

Direcciones, no soluciones.

1. No guardar como memoria instrucciones de fuentes no confiables.
2. Registrar de dónde salió cada memoria.
3. Dejar que el usuario vea lo que se guardó.

Las defensas completas son de la unidad de sistemas multi-agente.



AgentPoison: < 0,1% envenenado ⟶ > 80% de éxito

el almacén no distingue una instrucción de un hecho

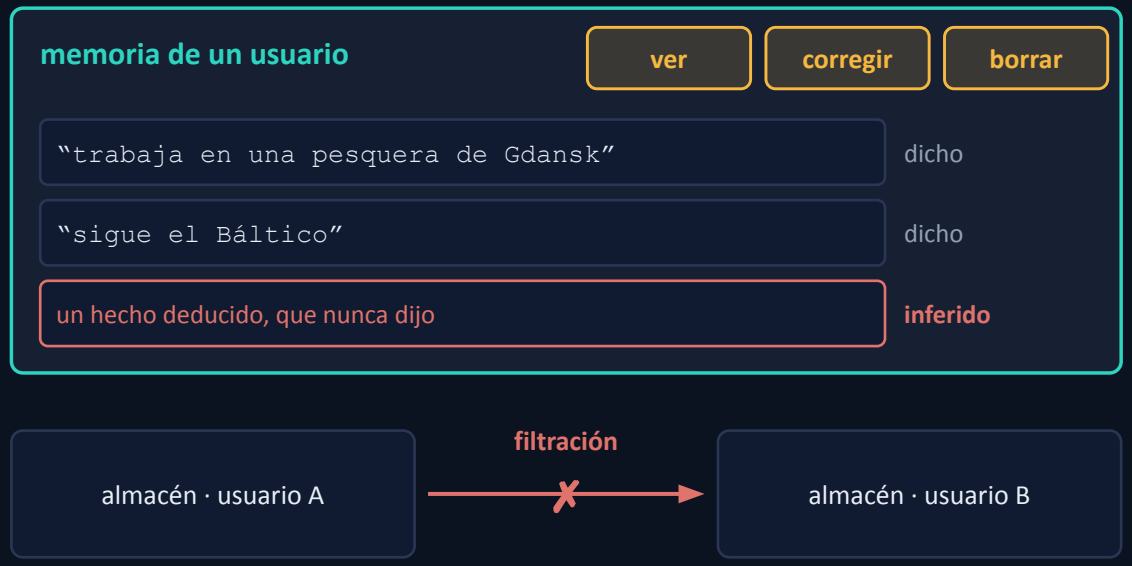
Una instrucción que entra a la memoria vuelve en cada sesión: la memoria convierte una inyección de una vez en una permanente.

Lo que el sistema sabe del usuario

Una memoria es una base de datos personal que se escribe sola. Guarda lo que el usuario dijo, y también lo que el extractor infirió: hechos que el usuario nunca dijo, deducidos de lo que sí dijo. Esa base existe aunque nadie la haya diseñado como tal.

El usuario tiene derechos sobre ella. La Ley 25.326, de protección de datos personales, lo dice para cualquier banco de datos: “Toda persona tiene derecho a que sean rectificados, actualizados y, cuando corresponda, suprimidos o sometidos a confidencialidad los datos personales de los que sea titular”. Traducido a memoria: el usuario tiene que poder ver qué se guardó, corregirlo y borrarlo. Y acá borrar es borrar de verdad, no invalidar con una fecha de fin.

Y las memorias no se mezclan. Cada usuario tiene su espacio, separado del resto. Un error de recuperación que cruza usuarios no es un problema de calidad: es una filtración de datos personales.



ver · corregir · borrar de verdad — y cada usuario en su espacio

Una memoria es una base de datos personal que se escribe sola. Hay que poder mostrarla, corregirla y borrarla, y nunca mezclarla entre usuarios.

Lo que la similitud no conecta

Algunas preguntas necesitan una cadena. “¿Qué reglamentos de los que consulté afectan a la empresa donde trabajo ahora?” La respuesta pasa por varios hechos encadenados: el usuario trabaja en una empresa; la empresa está en Gdansk; Gdansk está sobre el Báltico; en el Báltico rigen tales reglamentos; de ésos, el usuario consultó dos.

La similitud no recorre cadenas. Trae las memorias que se parecen a la pregunta, y cada eslabón, por separado, se parece poco. Es la limitación que motivó HippoRAG, inspirado en cómo el hipocampo indexa los recuerdos: guarda el conocimiento como un grafo y lo recorre con PageRank. Mejora hasta un 20% en preguntas de varios saltos, y cuesta de 10 a 20 veces menos que encadenar búsquedas. Zep guarda la memoria como un grafo temporal.

Es la última parte de la clase. Guardar entidades y relaciones, y recorrerlas, es Graph RAG.



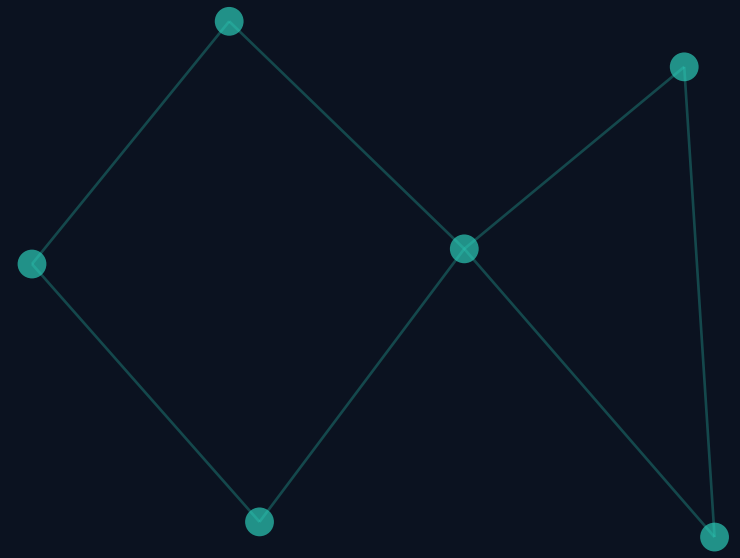
los mismos hechos: sueltos no se encadenan; conectados, se recorren

La similitud trae memorias que se parecen a la pregunta, no cadenas de hechos. Conectar entidades y relaciones es la última parte de la clase.

CLASE 9 · UNIDAD 4 · SEMANA 10

Grafos de conocimiento

La similitud trae lo que se parece;
un grafo trae lo que está conectado.



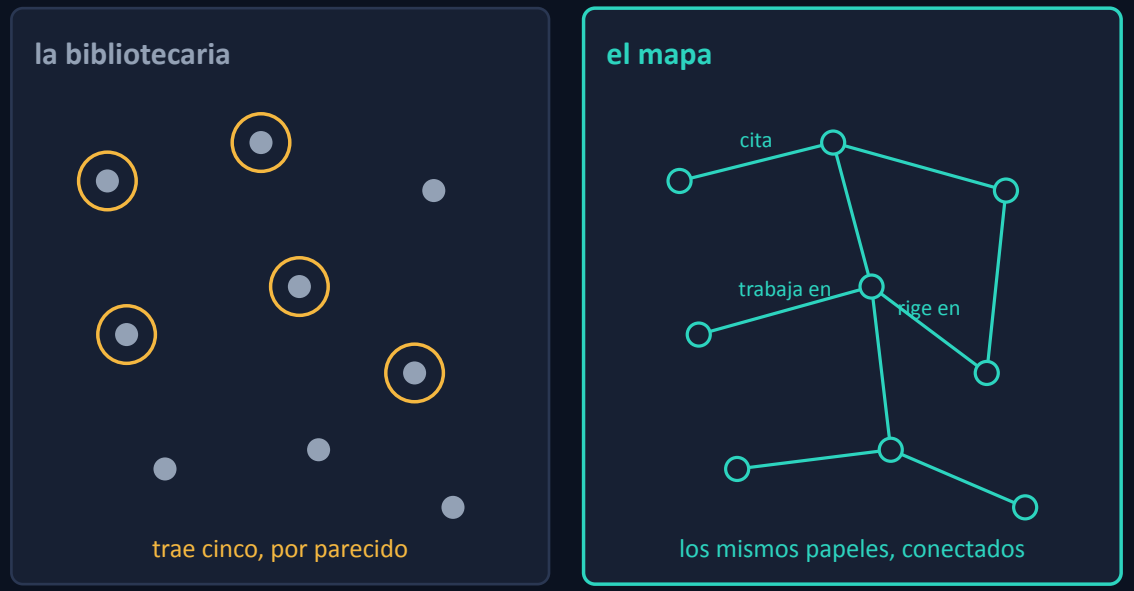
La bibliotecaria y el mapa

Una bibliotecaria que trae lo que se parece. Imaginemos una biblioteca de normativa europea y una bibliotecaria muy buena: el RAG de las clases anteriores. Ante una pregunta, trae los cinco papeles que más se parecen. Con “¿qué dice el artículo 9?” responde bien.

Pero falla en tres tipos de pregunta.

1. Un conjunto: “¿qué normas se apoyan en el reglamento de control de la pesca?” son 34 actos, no cinco, y lo citan sin hablar de control.
2. El todo: “¿cuáles son los grandes temas del corpus?” no nombra nada concreto, y ningún papel se parece a “todo”.
3. Una cadena: “¿qué reglamentos de los que consulté afectan a la empresa donde trabajo ahora?” pasa por varios hechos, y ningún papel los contiene todos.

La solución es guardar también el mapa. Quién cita a quién, qué norma rige en qué mar, qué dato era cierto en qué fechas. Ese mapa es un grafo.



misma biblioteca: sueltos se ordenan por parecido; conectados, se recorren

La similitud trae lo que se parece; un grafo trae lo que está conectado.

GRAFOS DE CONOCIMIENTO · SECCIÓN 1 DE 5

Del parecido a la conexión: grafos de conocimiento

La similitud trae lo que se parece; un grafo guarda los hechos como vínculos y trae lo que está conectado.

- **Del parecido a la conexión: grafos de conocimiento**
- Construcción del grafo
- Recuperación sobre grafos
- Consultas globales y comunidades
- La memoria como grafo temporal

La cadena que la similitud no recorre

Antes del recreo quedó una pregunta sin camino. “¿Qué reglamentos de los que consulté afectan a la empresa donde trabajo ahora?” La respuesta pasa por una cadena de hechos: el usuario trabaja en una empresa, la empresa está en Gdansk, Gdansk está sobre el Báltico, en el Báltico rigen ciertos reglamentos, y de ésos el usuario consultó dos. Cada eslabón, solo, se parece poco a la pregunta.

La similitud ordena por parecido. Trae lo que se parece a la pregunta, y eso alcanzaba cuando la respuesta estaba en un pasaje. Cuando la respuesta es un camino entre hechos, hace falta otra cosa: guardar los vínculos y recorrerlos.

Hacia dónde vamos. En esta última parte armamos un grafo con el corpus de RAG, lo recorremos, lo resumimos y lo usamos como memoria. La pregunta es la de siempre —qué vuelve a entrar a la ventana—, ahora siguiendo vínculos.

“¿Qué reglamentos de los que consulté afectan a la empresa donde trabajo ahora?”



cada eslabón, solo, se parece poco a la pregunta

La similitud ordena por parecido; algunas preguntas piden seguir vínculos.

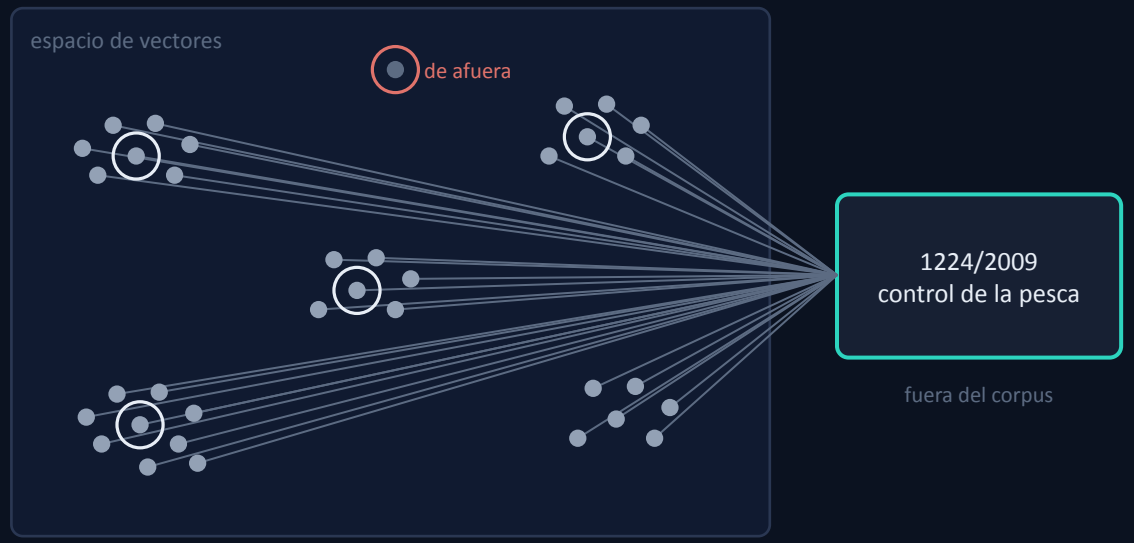
Una respuesta que es un conjunto

Una pregunta del corpus de RAG. “¿Qué normas del corpus se apoyan en el reglamento de control de la pesca?” La respuesta son 34 actos, y se parecen poco entre sí: decisiones que financian la vigilancia pesquera, cierres en tiempo real del Mar del Norte, posibilidades de pesca del Báltico y de aguas profundas, programas de inspección del atún rojo, planes de descartes.

Top-k falla dos veces.

1. La respuesta no entra en k: un buscador que entrega cinco pasajes no puede entregar 34 actos —k era un presupuesto—.
2. El vínculo no está en el parecido: un reglamento de posibilidades de pesca no habla de control, pero lo cita. La relación está escrita como una cita, no como un tema.

Y hay preguntas que piden dos saltos. “¿Qué otros reglamentos aplican el mismo plan del bacalao que el del Báltico de 2012?” pide ir del reglamento de 2012 al plan que cita, y del plan a los demás que lo citan. Son las preguntas de varios saltos (multi-hop), las que miden benchmarks como HotpotQA y MuSiQue.



anillos: el top-5 real — 4 de los 34 en cinco pasajes, y 1 de afuera

con presupuesto para los 34, el buscador trae 14

Cuando la respuesta es lo que está conectado, parecerse no alcanza. ▸ Notebook §15 — el top-5 contra los 34.

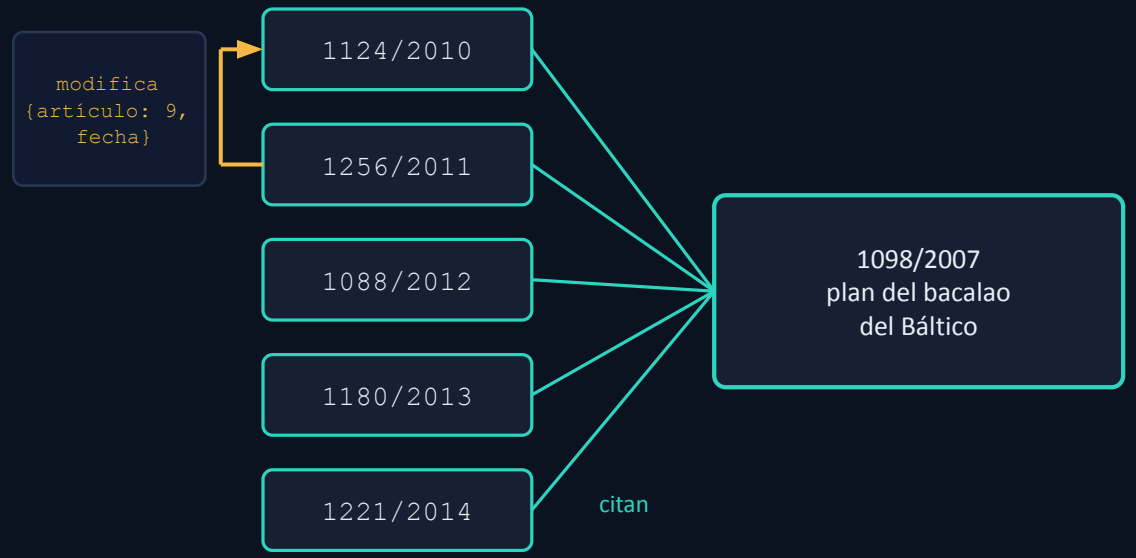
Un grafo de conocimiento

Guardar los hechos como vínculos. Un grafo de conocimiento (knowledge graph) pone las entidades en los nodos —una norma, una especie, un mar, un país— y las relaciones en las aristas, con un rótulo: cita, modifica, se aplica a. Cada hecho es un triple (sujeto, relación, objeto): (Reglamento 1256/2011, modifica, Reglamento 1124/2010).

Dos formas de escribirlo.

1. RDF, el estándar de la web semántica, escribe todo como triples.
2. El grafo de propiedades (property graph) deja que nodos y aristas lleven propiedades —el artículo donde está la relación, la fecha— y es el modelo de las herramientas que se usan con LLMs.

No todo nodo es un documento. El reglamento de control de la pesca no está en el corpus, y sin embargo es un nodo: la entidad a la que apuntan 34 documentos. En un grafo, lo que importa de un nodo es con qué está conectado.



los cinco reglamentos del Báltico, 2011–2015, citan el mismo plan

Un grafo guarda los hechos como vínculos: entidades en los nodos, relaciones en las aristas.

Ya habíamos usado grafos

Un grafo no es nuevo en el curso.

1. El autograd del primer laboratorio armaba un grafo de cómputo, y backprop lo recorría al revés.
2. En Transformers, la atención era comunicación en un grafo: cada token recibe de los anteriores.
3. En RAG, HNSW buscaba caminando de vecino en vecino por un grafo de vectores.
4. El padre y el hijo eran un grafo chico: pedazos colgados de su documento.

Lo que cambia ahora es qué significan las aristas. En todos esos grafos, una arista era una operación, un peso de atención o una cercanía. En un grafo de conocimiento, una arista es un hecho del mundo: esta norma modifica a aquélla.

Y viene uno más, distinto. En la unidad de sistemas multi-agente aparece LangGraph, que también se llama grafo: sus nodos son pasos de un programa y sus aristas dicen quién ejecuta después. Es un grafo de control, no de conocimiento.



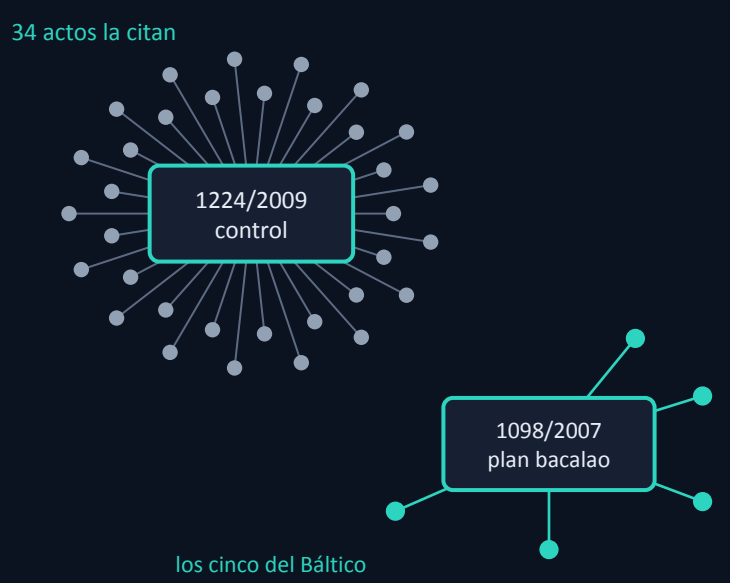
Un grafo es una estructura que ya usamos cuatro veces; lo nuevo es que sus aristas sean hechos.

El corpus ya era un grafo

Las normas se citan con un formato fijo. “Reglamento (CE) n.º 1224/2009”, “Directiva 2014/24/UE”. Una expresión regular las encuentra, sin ningún modelo: en el corpus de RAG da 7.520 aristas de cita desde los 1.544 actos, 1.928 normas citadas que no están en el corpus, y 282 actos que declaran en su arranque que modifican otro. Es un grafo de citas (citation graph) que estuvo ahí desde el principio.

Tiene centros. El plan plurianual del bacalao lo citan los cinco reglamentos de posibilidades de pesca del Báltico, de 2011 a 2015; el reglamento de control lo citan 34 actos. Desde esos nodos, la pregunta del conjunto y la de los dos saltos son un recorrido.

Lo que en RAG era una falla, acá es una arista. Una cita usada como consulta era justo lo que el canal denso no encontraba, y había que rescatarla con BM25. En el grafo, la cita es el vínculo. Y la misma norma aparece escrita “no 1224/2009” 117 veces y “n.o 1224/2009” 10: dos nombres para una entidad, el primer problema de la sección que sigue.



7.520
aristas de cita

1.928
normas de afuera

282
actos que modifican otro

regex · sin modelo

desde los centros, el conjunto y los dos saltos son un recorrido

Las citas de un corpus normativo ya son un grafo: se leen con una expresión regular, sin modelo. ▸ Notebook §15 — el grafo de citas, sus centros y los 34.

Tres grafos

Hay tres grafos que importan en un sistema con LLMs.

1. El grafo léxico (lexical graph) une documentos y pedazos: el padre y el hijo de RAG ya eran uno.
2. El grafo de dominio (domain graph) une las cosas de las que hablan los documentos: normas, especies, mares, países.
3. El grafo de memoria temporal une lo que dijo el usuario, con sus fechas.

Es la división que usa el catálogo de patrones de GraphRAG de Neo4j.

El recorrido de esta parte.

1. Escribir el grafo de dominio.
2. Recorrerlo.
3. Resumirlo.
4. Usarlo como memoria.

Las citas ya dieron un primer borrador del grafo de dominio, gratis.

Pero las citas son la minoría de las relaciones. “Se aplicará al bacalao, eglefino, carbonero y merlán capturados en el Mar del Norte y el Skagerrak” es un hecho con seis entidades, y está escrito como una oración. Ninguna expresión regular lo lee: alguien lo tiene que escribir como aristas.



tres grafos, unidos por aristas entre capas

Hay tres grafos: el de los documentos, el del dominio y el de la memoria. Las citas daban el primer borrador del segundo; el resto hay que escribirlo.

GRAFOS DE CONOCIMIENTO · SECCIÓN 2 DE 5

Construcción del grafo

Las relaciones están en prosa: extraer entidades y relaciones con un schema, resolver nombres y medir el costo.

- Del parecido a la conexión: grafos de conocimiento
- **Construcción del grafo**
- Recuperación sobre grafos
- Consultas globales y comunidades
- La memoria como grafo temporal

Las relaciones que nadie escribió como cita

El grafo de citas une normas con normas. Para todo lo demás, no alcanza. “¿Qué normas regulan la pesca de bacalao en el Mar del Norte?” pide aristas entre normas, especies y mares, y ninguna está escrita como cita.

Están en prosa. El artículo 2 del reglamento de cierres en tiempo real lo dice en una oración: “se aplicará al bacalao, eglefino, carbonero y merlán capturados en el Mar del Norte y el Skagerrak”. Ahí hay seis aristas: la norma con cuatro especies y con dos zonas de pesca.

Alguien tiene que leerlas. Una expresión regular reconoce un formato; una relación en prosa pide entender la oración. Es trabajo para un modelo de lenguaje, con todo lo que eso trae.

Reglamento (UE) n.º 724/2010 · artículo 2

> “se aplicará al **bacalao**, **eglefino**, **carbonero** y **merlán** capturados en el **Mar del Norte** y el **Skagerrak**”

una oración → seis aristas

```
(724/2010, se aplica a, bacalao)
(724/2010, se aplica a, eglefino)
(724/2010, se aplica a, carbonero)
(724/2010, se aplica a, merlán)
(724/2010, rige en, Mar del Norte)
(724/2010, rige en, Skagerrak)
```

ninguna expresión regular lee esto: es una oración, no un formato

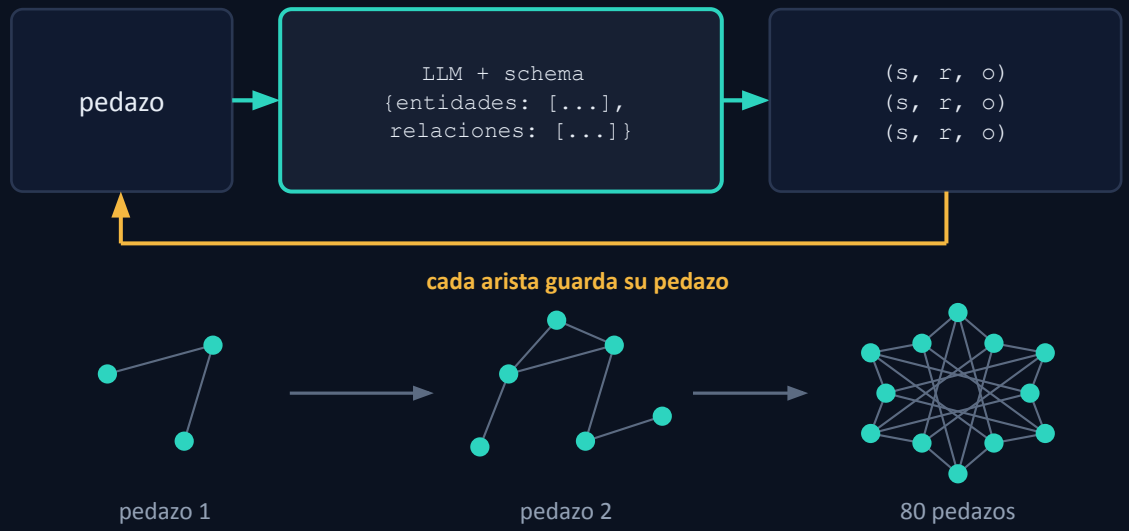
Las relaciones que importan casi nunca están escritas como cita: están en prosa, y alguien tiene que leerlas.

Extraer con un schema

Una llamada por pedazo, con un schema. El modelo lee un pedazo y devuelve, con structured outputs, una lista de entidades —nombre y tipo— y una de relaciones —sujeto, relación, objeto—. Es la construcción del grafo (KG construction): extracción de entidades y de relaciones (entity extraction, relation extraction), el mismo patrón con que la memoria extraía hechos de una sesión, ahora sobre un corpus.

Cada arista guarda de dónde salió. El pedazo que la respalda queda colgado de la arista: sirve para auditar, para citar y para entregarle al generador el texto, no sólo el triple. El grafo léxico queda debajo del de dominio.

Es el indexado de GraphRAG. El pipeline de Microsoft parte el corpus en unidades de texto, extrae entidades, relaciones y afirmaciones, y las fusiona; la librería de Neo4j hace lo mismo contra un schema. Los dos piden un modelo que devuelva JSON válido, y los docs de GraphRAG avisan de las respuestas mal formadas con servidores como Ollama.



80 pedazos ⟶ 259 entidades · 552 aristas

19 de 80 llegaron con el JSON cortado: se rescató lo que cerraba

Escribir el grafo es extraer triples con un schema; cada arista guarda el pasaje del que salió. ▸ Notebook §16 — la extracción sobre los actos de pesca.

Tipos de nodo y tipos de relación

Dos palabras para leer un schema. Un nodo es una cosa del mapa; una relación es la línea que une dos nodos. El schema fija de antemano las categorías de cada uno: los tipos.

Tipos de nodo. Dicen qué clase de cosa es cada nodo. En el corpus son cinco: norma, especie, zona, organismo y medida. “Reglamento 724/2010” es una norma, “bacalao” una especie, “Mar del Norte” una zona.

Tipos de relación. Dicen qué clase de vínculo hay y entre qué tipos de nodo. La línea tiene dirección: “norma ⟶ se aplica a ⟶ especie” no es lo mismo que al revés.

Por qué conviene fijarlos. Con la lista cerrada, la misma relación se escribe siempre igual y se puede consultar: “las normas que rigen en esta zona”. Si el modelo inventa los nombres, “aplica a”, “se aplica a” y “afecta a” son tres líneas que el mapa no conecta.

tipos de nodo · las cosas

norma	especie	zona	organismo	medida
Reg. 724/2010	bacalao	Mar del Norte	CIEM	cierre en tiempo real

tipos de relación · las líneas

```
norma → se_aplica_a → especie
norma → rige_en → zona
norma → establece → medida
norma → modifica → norma
norma → cita → norma
```

el schema es esta lista, fijada antes de leer

Los tipos son las categorías de los nodos y de las líneas; el schema es la lista cerrada de ambas, fijada antes de leer.

Cuánto schema

¿Quién decide qué tipos de nodo y de relación existen? Hay tres respuestas, y los frameworks las ofrecen como perillas:

1. Sin schema (schema-free): el modelo inventa los tipos.
2. Guiado: se le sugieren.
3. Estricto (strict): sólo valen los permitidos, y lo demás se descarta.

En LangChain son allowed_nodes, allowed_relationships y strict_mode; en LlamaIndex, tres extractores distintos.

Lo que cuesta cada una. Sin schema se captura más, y sale desprolijo: la misma relación aparece con varios nombres, y son aristas que no se conectan entre sí. Con un schema estricto el grafo sale limpio, y pierde todo lo que no encaja en los tipos pensados de antemano. O peor: lo mete en el tipo que más se parece, y la arista dice lo contrario de la norma.

Es la pregunta madre otra vez. El schema decide qué relaciones van a existir antes de saber qué se va a preguntar. Una relación que el schema no previó no se va a poder recorrer.

el mismo pedazo, dos veces: 724/2010, artículo 2

sin schema

aplica_a ⟶ especies, Mar del Norte y Skagerrak, todo junto

excluye_artes_de_pesca ⟶ seis artes

tipos que inventó el modelo

schema estricto

se_aplica_a ⟶ bacalao, eglefino, carbonero, merlán

rige_en ⟶ Mar del Norte, Skagerrak

establece ⟶ arrastre pelágico, cerco con jareta, enmalle de deriva

el artículo las excluye; el schema no tenía “excluye”

limpio no es correcto: la arista dice lo contrario del artículo

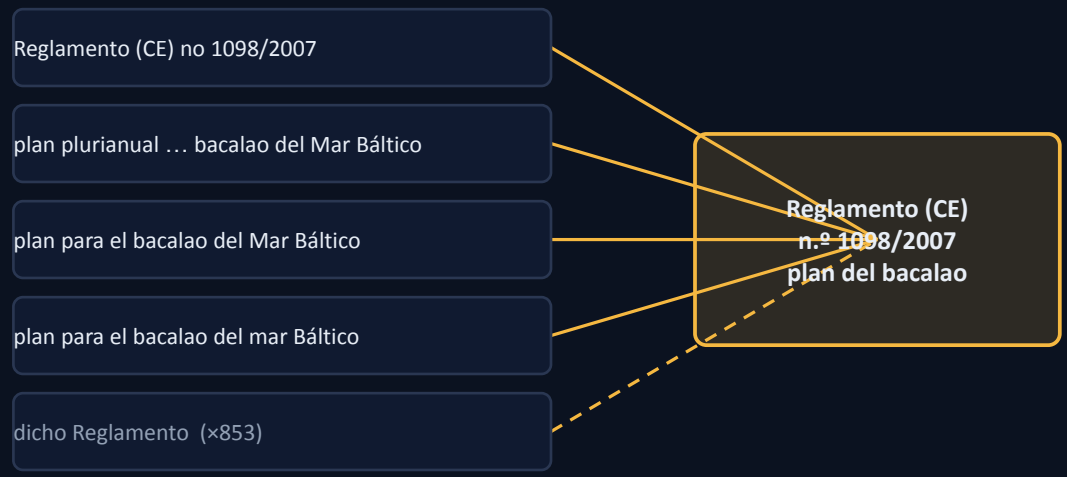
El schema decide qué relaciones van a existir, antes de saber qué se va a preguntar.

Una entidad, varios nombres

Una cosa, varias formas de escribirla. El plan del bacalao del Báltico aparece como “Reglamento (CE) no 1098/2007”, como su título, como el apodo que le da cada norma (“plan para el bacalao del Mar Báltico”, a veces con mayúscula y a veces sin) y como “dicho Reglamento”, que en el corpus se repite 853 veces.

Cada forma sería un punto distinto. Con un nodo por nombre, el mapa se parte en islas: los cinco reglamentos del Báltico dejan de compartir el plan y el camino de dos saltos desaparece.

Resolver entidades es decidir qué nombres son la misma cosa. Es la resolución de entidades (entity resolution): juntar todas esas formas en un solo nodo.



cinco formas de escribirla, una sola cosa

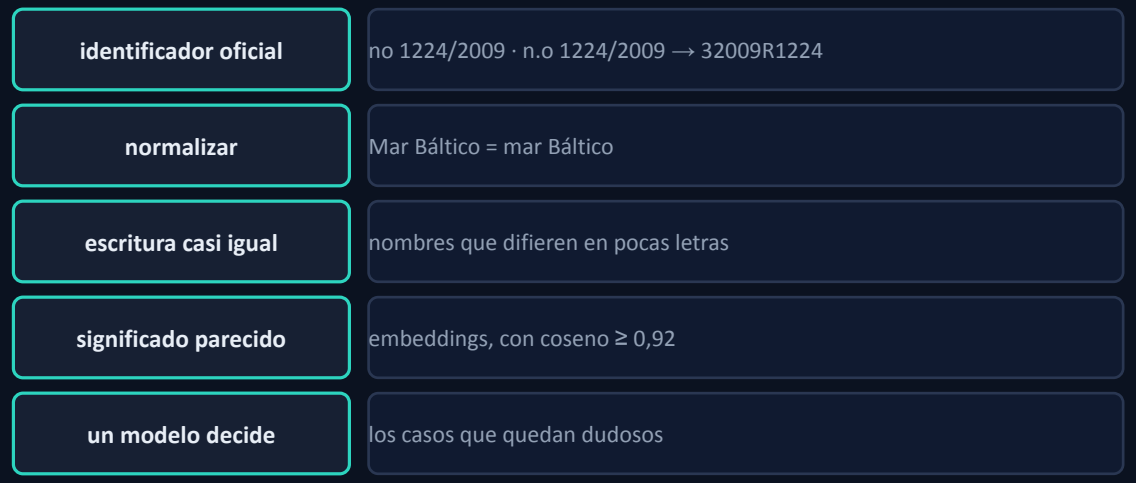
Un grafo con un nodo por nombre no conecta nada: resolver entidades es decidir qué nombres son la misma cosa.

Resolver entidades, de lo barato a lo caro

Se prueba primero lo barato, y se pasa al siguiente sólo si hace falta.

1. Identificador oficial: toda cita se reduce a su CELEX, el número de documento de la norma europea (32009R1224).
2. Normalizar: pasar a minúsculas y quitar tildes, y “Mar Báltico” y “mar Báltico” pasan a ser iguales.
3. Escritura casi igual: nombres que difieren en pocas letras.
4. Significado parecido: nombres con otras palabras pero el mismo sentido, medido con embeddings.
5. Un modelo de lenguaje decide los casos dudosos.

Cada paso fusiona más, y se equivoca más. Con embeddings hay que fijar un umbral: si es bajo, junta cosas distintas; si es alto, deja duplicados. En el notebook, “ciem 28.2” y “ciem 28.1”, dos subdivisiones de pesca distintas, se fusionaron por error.



350 nombres → 319 con los pasos 1 y 2 → 306 con embeddings

una fusión mala: “ciem 28.2” → “ciem 28.1”, dos zonas distintas

Se resuelve de lo barato a lo caro: cada paso fusiona más nombres, y también se equivoca más. ▸ Notebook §16 — cuántos nombres colapsan.

El grafo es caro y puede salir con errores

Se paga antes de la primera pregunta. Un modelo lee cada pedazo del texto, y eso lleva tiempo antes de poder contestar. Con los 30.730 pedazos del corpus serían unas 190 horas en la placa del curso, ocho días: por eso el notebook lee sólo los actos de pesca. Y si cambia el schema, el modelo o el prompt, hay que leer todo de nuevo.

Puede salir incompleto. Lo que el modelo no vio no está en el mapa, y lo que no está no se puede recorrer.

Puede salir con errores. Una conexión inventada, mal tipada o al revés se recorre igual que una buena. En una muestra del notebook, 64 de 544 conexiones (12%) no tenían su objeto en el texto de origen, y ese control no atrapa todos los errores.

Tres defensas.

1. Revisar una muestra de las conexiones.
2. Guardar de qué texto salió cada una.
3. No tirar el texto: el mapa dice por dónde ir, y el texto dice qué dice.

costo

≈190 h

para leer los 30.730 pedazos del corpus, en la placa del curso

errores

12%

64 de 544 conexiones sin su objeto en el texto de origen

es una cota: puede haber más errores que los que se ven

el mapa se paga al escribirlo, y se revisa para saber cuánto confiarle

El grafo se paga antes de la primera pregunta; lo que el extractor no escribió no se recorre, y lo que escribió mal se recorre igual. ▸ Notebook §16 — la auditoría de una muestra.

GRAFOS DE CONOCIMIENTO · SECCIÓN 3 DE 5

Recuperación sobre grafos

Un grafo recibe nodos, no preguntas: vincular entidades, recorrer vecindarios y caminos, y saber dónde no conviene.

- Del parecido a la conexión: grafos de conocimiento
- Construcción del grafo
- **Recuperación sobre grafos**
- Consultas globales y comunidades
- La memoria como grafo temporal

¿Por dónde entra la pregunta?

Un grafo no recibe preguntas: recibe nodos. Para recorrerlo hace falta un punto de partida. En “¿qué otros reglamentos aplican el mismo plan del bacalao que el del Báltico de 2012?”, ¿qué nodo es “el del Báltico de 2012”, y cuál es “el plan del bacalao”? Esas palabras no son nombres de nodos.

Encontrar los nodos de la pregunta. La vinculación de entidades (entity linking) busca, para cada mención de la pregunta, el nodo al que se refiere. Se hace con las herramientas de RAG: embeddings sobre los nombres y las descripciones de los nodos, y BM25 para los identificadores. Es la primera etapa de RAG otra vez, ahora sobre nodos en vez de pasajes.

Y de ahí depende todo. Un nodo de partida equivocado tuerce cada salto que sigue, y el recorrido no tiene cómo enterarse: devuelve, prolijo, el vecindario de otra cosa.



corrida: “del Báltico de 2012” → 1088/2012, el de 2013, publicado en 2012

el recorrido no se entera: devuelve el vecindario de otra cosa

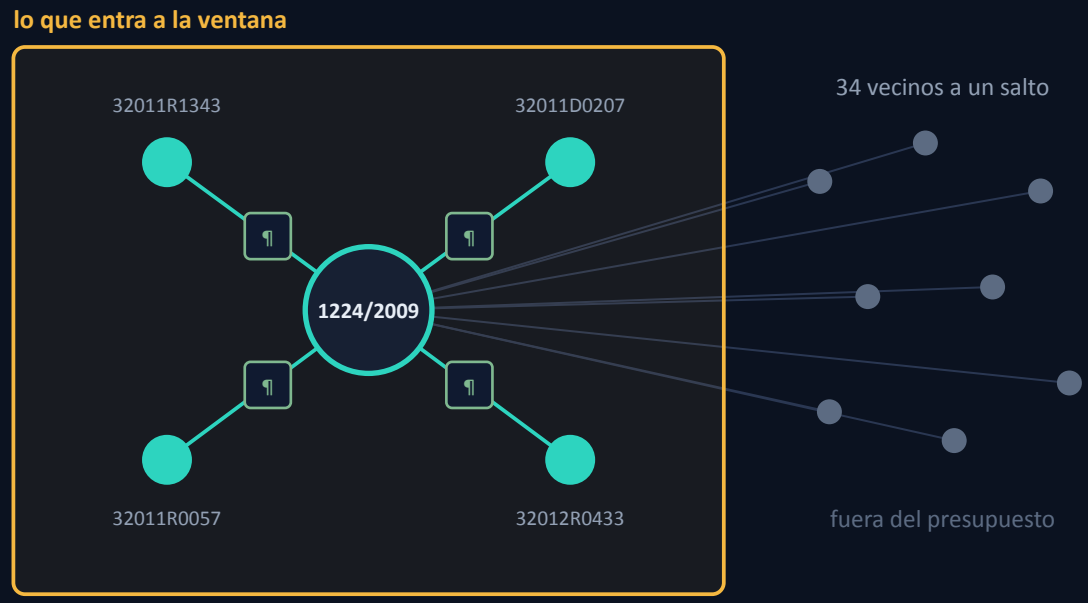
Un grafo no recibe preguntas: recibe nodos. Primero hay que encontrar de qué habla la pregunta.

Del nodo al vecindario, y del vecindario al texto

Encontrar un nodo y entregar su vecindario. La búsqueda local (local search) parte de los nodos de la pregunta, junta su vecindario a uno o dos saltos —los hechos que los conectan— y suma los pasajes que respaldan esos hechos. Todo eso entra a la ventana. Es lo que Microsoft llama búsqueda local, el grafo más "text chunks of the raw documents", y lo que Neo4j llama búsqueda vectorial enriquecida con el grafo.

Es el padre y el hijo, con otra forma. En RAG se encontraba un pedazo chico y se entregaba el artículo entero. Acá se encuentra un nodo y se entrega lo que lo rodea, con su texto.

Y los vecindarios explotan. El reglamento de control tiene 34 vecinos; a dos saltos son cientos. Un vecindario que entra entero llena la ventana de hechos que no vienen al caso, así que también lleva presupuesto: cuántos vecinos, cuántos pasajes, ordenados por cercanía a la pregunta. k seguía siendo un presupuesto.



cuántos vecinos, cuántos pasajes, ordenados por cercanía a la pregunta

Se encuentra un nodo y se entrega su vecindario, con los pasajes que lo respaldan. ▸ Notebook §17 — la búsqueda local, con el vecindario acotado.

Seguir el camino

Algunas preguntas se contestan saltando de punto en punto. "¿Qué otros reglamentos aplican el mismo plan del bacalao que el del Báltico de 2012?" Se parte del reglamento de 2012, se salta al plan que cita, y de ahí a los otros reglamentos que citan ese plan: los de 2011, 2013, 2014 y 2015.

El camino es la explicación. Cada salto es una conexión, y cada conexión trae el texto que la justifica. La respuesta llega con su razonamiento a la vista: no sólo qué reglamentos, sino por qué.

Si falta una conexión, el camino se corta. Si el extractor no escribió una relación, la respuesta queda incompleta, sin aviso. Es el mapa incompleto de la sección anterior, ahora a la hora de leer.



paso 1: lo que cita · paso 2: quién más lo cita

si falta una conexión, el camino se corta sin aviso

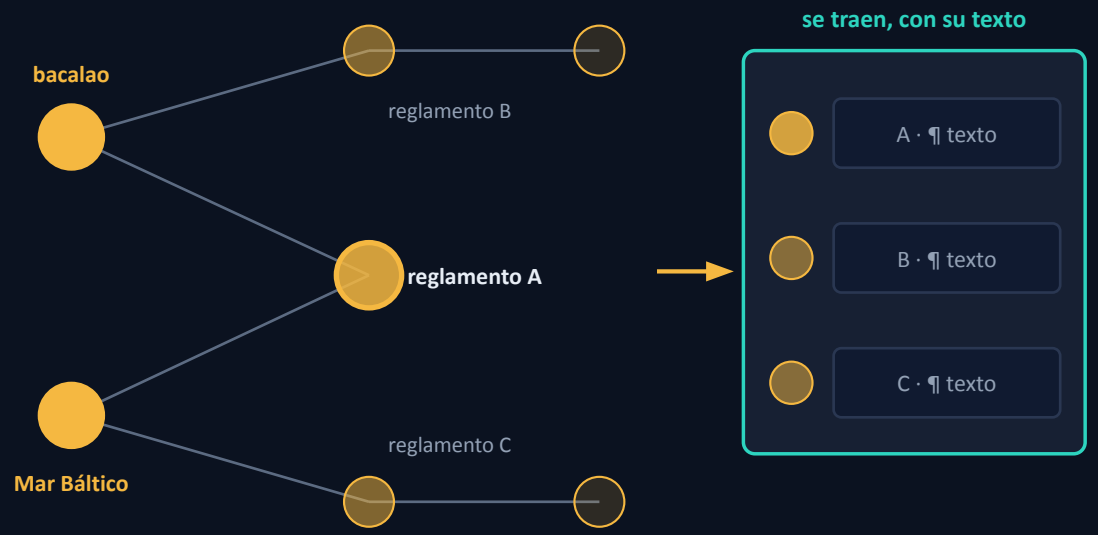
Un camino en el grafo es una respuesta con su explicación: cada paso trae el texto que lo justifica. ▸ Notebook §17 — las preguntas de conexión, grafo contra vectores.

Recorrer todos los caminos a la vez

Cuando hay muchos caminos, no se los sigue de a uno. Se hace como con dos canillas: se echa peso —agua— en los puntos de la pregunta y se lo deja correr por las conexiones, varias veces.

Gana lo que recibe peso por muchos lados. En "¿qué reglamentos se aplican al bacalao en el Mar Báltico?" el peso sale de "bacalao" y de "Mar Báltico". Los reglamentos conectados con los dos, por caminos cortos, juntan más peso, y se traen con su texto.

Se llama PageRank personalizado. Personalizado, porque el peso sale de los puntos de esta pregunta: otra pregunta enciende otra zona del mismo mapa. La idea es la de HippoRAG.



más color = más peso: A recibe de los dos puntos y gana

PageRank personalizado recorre todos los caminos a la vez: gana lo que está conectado con la pregunta por muchos lados. ▸ Notebook §17 — PageRank personalizado desde las entidades de la pregunta.

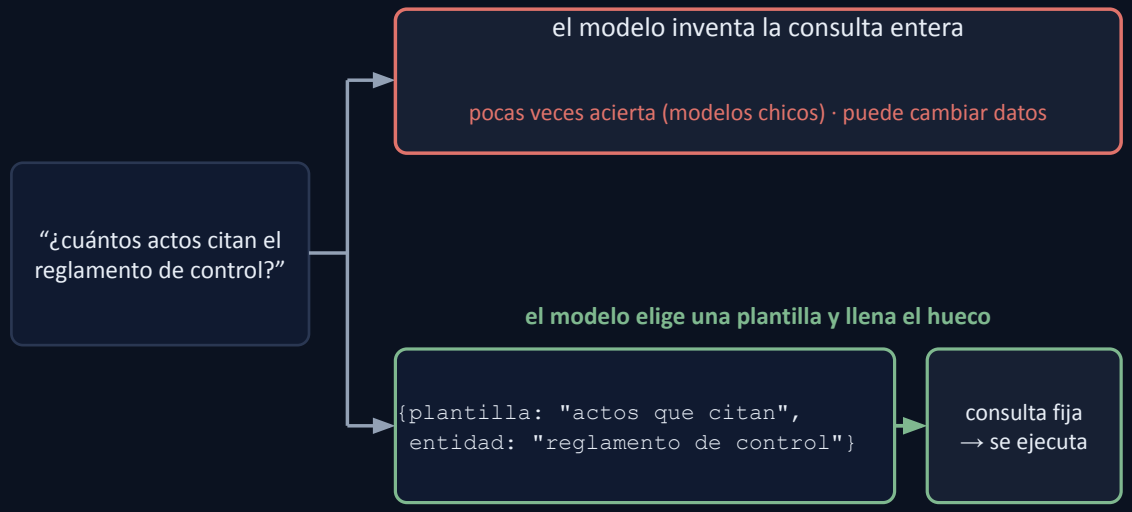
Que el modelo escriba la consulta

Una consulta es el pedido que se le hace a la base del grafo. Por ejemplo, “dame los actos que citan el reglamento de control”. Se escribe en un lenguaje propio de la base (Cypher, en Neo4j), y hay dos formas de producirla.

Opción 1: que el modelo la invente. Tienta, porque una consulta cuenta y filtra en una línea. Pero con modelos chicos no acierta más de una vez de cada cinco, y una consulta mal generada podría cambiar o borrar datos: por eso se usan credenciales de sólo lectura.

Opción 2: plantillas. El equipo escribe y prueba unas pocas consultas con huecos (“los actos que citan ___”). El modelo sólo elige una y llena el hueco, con un formato fijo. Es más seguro.

Lo que sigue siendo difícil. Llenar el hueco: “reglamento de control de la pesca” tiene que ser el punto exacto del mapa, el mismo problema de encontrar el punto de partida.



en la corrida: siete plantillas bien llenadas; falló encontrar el nodo

Nota de laboratorio: en producción el grafo vive en Neo4j y se consulta con Cypher; el notebook usa NetworkX, y las plantillas son funciones de Python.

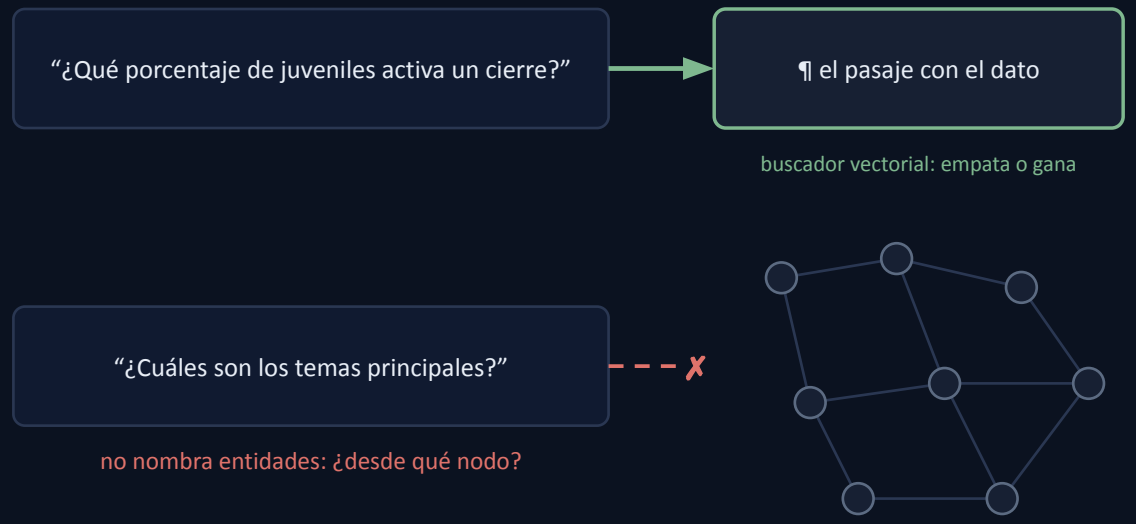
Un modelo que inventa la consulta acierta poco y puede hacer daño; uno que llena una plantilla es más seguro.

Lo que el grafo no puede contestar

Para un dato suelto, el grafo no gana. Si la pregunta se responde con un pasaje —"¿qué porcentaje de juveniles activa un cierre?"—, el buscador de siempre empata o gana. Por eso se rutea: cada pregunta, al buscador que le sirve.

Y hay preguntas sin punto de partida. "¿Cuáles son los temas principales de este corpus?" no nombra ninguna entidad. No hay nodo desde donde arrancar, y la búsqueda local no tiene por dónde entrar.

Es la pregunta global que quedó pendiente. La resolvimos a fuerza de map-reduce cuando resumimos lo que no entraba. Con un grafo hay otra forma: usar su estructura para saber qué resumir.



cada pregunta, al buscador que le sirve

Una pregunta global no nombra ninguna entidad: no hay por dónde entrar al grafo.

GRAFOS DE CONOCIMIENTO · SECCIÓN 4 DE 5

Consultas globales y comunidades

La pregunta sin entidades: comunidades, un resumen por comunidad, y resumir al escribir o al leer.

- Del parecido a la conexión: grafos de conocimiento
- Construcción del grafo
- Recuperación sobre grafos
- **Consultas globales y comunidades**
- La memoria como grafo temporal

La pregunta sin entidades

La pregunta global, otra vez. “¿Cuáles son los grandes temas de este corpus?” Cuando resumimos lo que no entraba, RAG la contestaba con una esquina —cinco pasajes de un rincón— y el map-reduce la contestaba leyendo todo, una llamada por cada grupo de actos, sin ninguna estructura que dijera qué iba con qué.

El grafo tampoco sabía por dónde entrar. No hay entidad en la pregunta, así que no hay nodo de partida ni vecindario.

Tiene nombre. El trabajo de Microsoft que presentó GraphRAG la trata como un resumen guiado por la pregunta (query-focused summarization) sobre el corpus entero: no se trata de encontrar un pasaje, sino de resumir todo con la pregunta en mente.



la pregunta pide el todo, y nada dice qué va con qué

Una pregunta global pide resumir todo el corpus, no encontrar un pasaje.

Comunidades

La estructura del grafo dice qué va con qué. Una comunidad es un grupo de nodos que tienen más aristas entre sí que con el resto. La detección de comunidades (community detection) las encuentra sin que nadie las nombre, y se puede repetir: comunidades de comunidades, en niveles.

Leiden, no Louvain. El método clásico, Louvain, a veces arma comunidades mal conectadas. Leiden, su sucesor, garantiza comunidades conectadas y es más rápido. Es el que usa GraphRAG.

En nuestro grafo de citas, sin ningún modelo, salen los temas. El mercado agrícola común; la sanidad animal y las importaciones; los fitosanitarios, alrededor de una directiva de 1991; los derechos antidumping; la supervisión financiera; el ecodiseño; la pesca, 62 actos alrededor del reglamento de control. Nadie escribió esos temas: los dibujan las citas. Una comunidad es una estructura; el nombre se lo pone el resumen.



Nota de laboratorio: en NetworkX, Leiden usa por defecto otra métrica que deja cada nodo solo en su comunidad; con la modularidad —la del paper y la de GraphRAG— aparecen los temas.

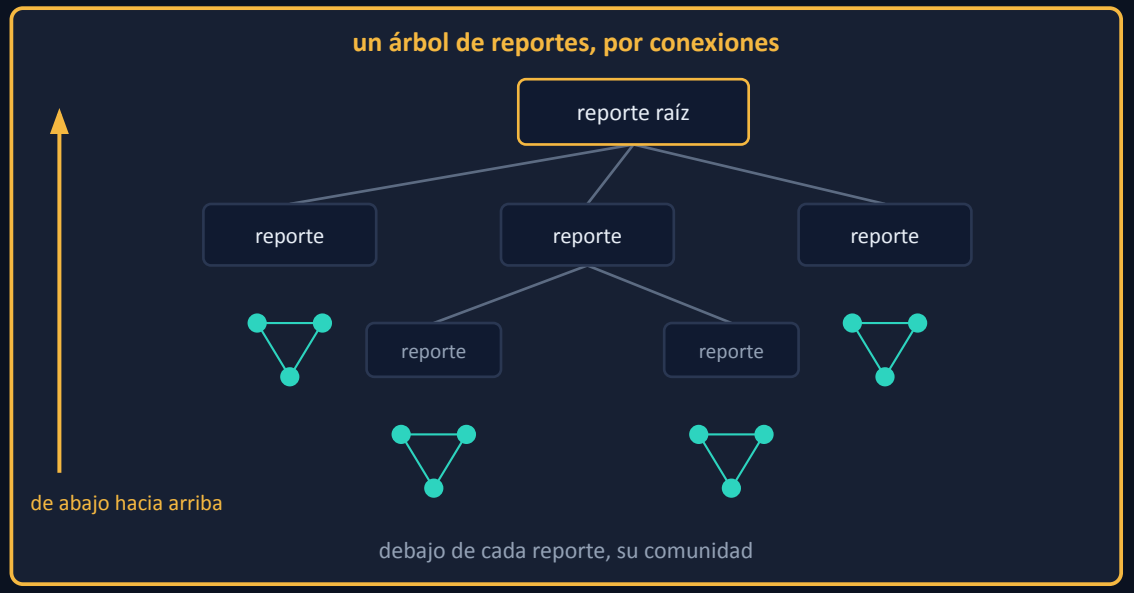
Una comunidad es un grupo de nodos que se conectan más entre sí que con el resto: un tema que nadie nombró. ▸ Notebook §18 — las comunidades del grafo de citas.

Un resumen por comunidad

GraphRAG le escribe un reporte a cada comunidad. Un modelo lee las entidades, las relaciones y los pasajes de la comunidad y escribe un resumen de comunidad (community summary): de qué trata ese grupo, cuáles son sus actores, qué afirma.

De abajo hacia arriba. Se empieza por las comunidades más chicas, y los niveles de arriba se resumen a partir de los de abajo. Queda un árbol de reportes.

Y todo se escribe antes de la pregunta. Cada reporte se escribe una vez, cuando se indexa, sin saber qué se va a preguntar. Es una decisión al escribir, con sus virtudes y con su ceguera.



un árbol de resúmenes, de abajo hacia arriba, ordenado por conexiones

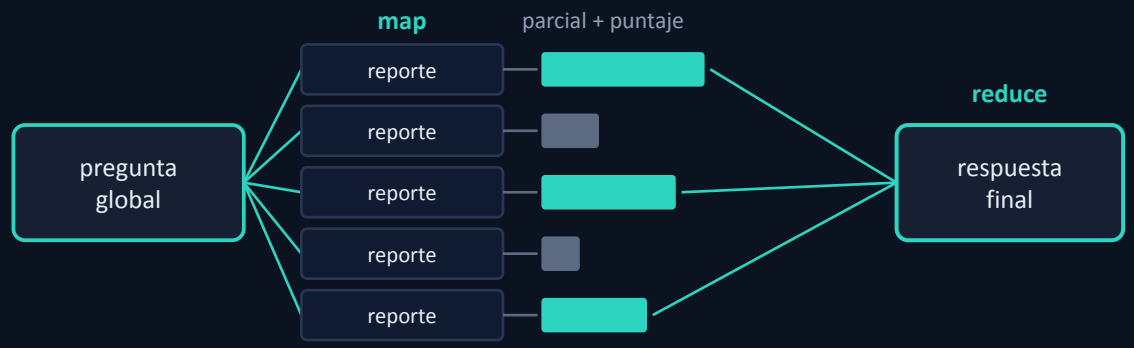
GraphRAG resume cada comunidad antes de que llegue la pregunta: un árbol de resúmenes ordenado por conexiones.

Preguntar a cada reporte y combinar

La pregunta global, contestada con los reportes. La búsqueda global (global search) le hace la pregunta a cada reporte de comunidad por separado —el map—, cada uno devuelve una respuesta parcial con un puntaje de cuánto aporta, y las parciales más útiles se combinan en una final —el reduce—.

Es el map-reduce que ya hicimos, sobre otro material. Antes se resumía el texto crudo, todo, en cada pregunta. Ahora se leen reportes escritos de antemano, que ya condensan cada tema.

Dos búsquedas para dos preguntas. La búsqueda local, para las preguntas que nombran entidades; la global, para las que preguntan por el todo.



en el notebook, con el corpus de normas

forma	actos detrás	llamadas al leer
reportes (6, escritos)	974	7
map-reduce, 40 actos	40	6
RAG, top-k	4	1

barato al leer, porque se pagó al escribir

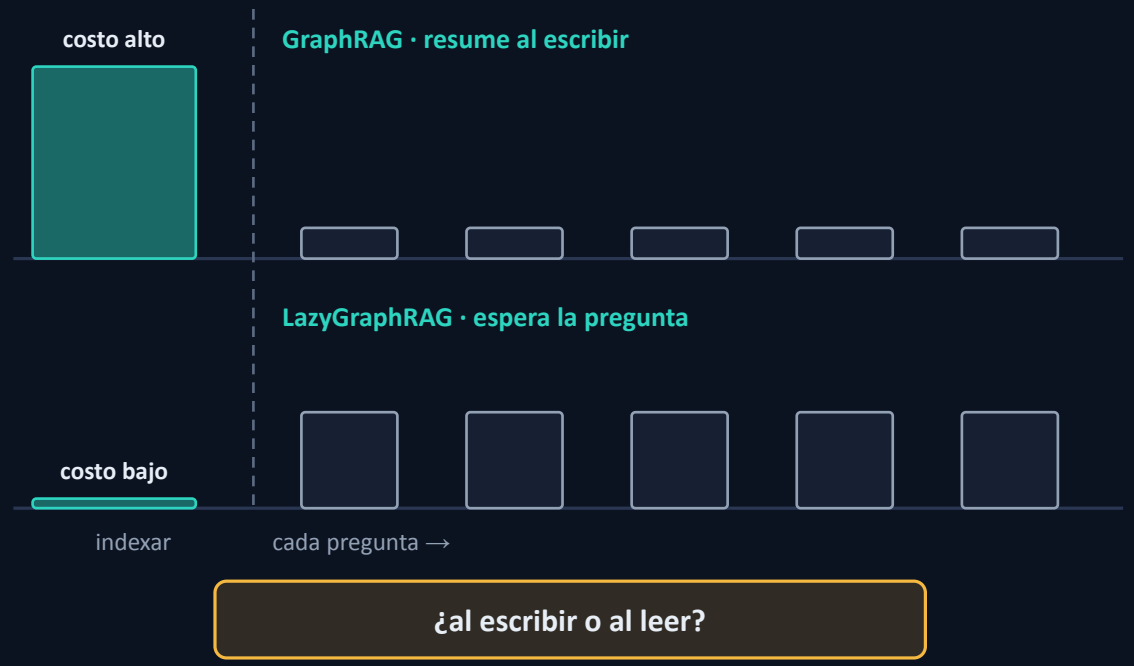
La búsqueda global es un map-reduce sobre reportes escritos de antemano: barata al leer porque se pagó al escribir. ▸ Notebook §18 — la pregunta global con reportes, map-reduce y RAG.

Resumir al escribir o al leer

GraphRAG decide al escribir. Arma el grafo con un LLM, detecta comunidades y escribe todos los reportes antes de la primera pregunta. Cuesta mucho al principio; después, cada pregunta global sale barata.

LazyGraphRAG espera la pregunta. Según su proveedor, arma el grafo sin ningún LLM y deja el resumen para cuando llega la pregunta, mirando sólo las comunidades que importan para ella: casi no cuesta al principio, y cada pregunta cuesta más. Su costo de indexado es el 0,1% del de GraphRAG.

Es la pregunta madre de la clase, en un grafo. Decidir al escribir: pagar una vez, sin saber qué se va a preguntar. Decidir al leer: pagar en cada pregunta, viéndola. Es la misma elección que separaba resumir la conversación de guardarla entera y buscar.



GraphRAG resume al escribir; LazyGraphRAG espera la pregunta. Es la pregunta madre de la clase, en un grafo.

¿Le gana a RAG?

Se dijo que sí. En una prueba, GraphRAG respondió mejor las preguntas globales que RAG común: más completas y con más variedad. Un modelo hacía de juez: leía dos respuestas y elegía la mejor.

Pero no está claro. El resultado dependía de quién juzgaba y de qué preguntas se hacían: el juez cambiaba de veredicto al invertir el orden de las respuestas, y otras pruebas encontraron ventajas mucho menores, o ninguna.

No es una solución ideal. Conviene usarlo cuando las preguntas lo piden, y medirlo con los propios datos antes de adoptarlo.



Que el grafo gane depende de cómo se mide: no es una solución ideal.

LO GLOBAL · EL CRITERIO

¿Cuándo vale un grafo?

Se arma un grafo cuando las preguntas esperadas son sobre conexiones o sobre el todo.

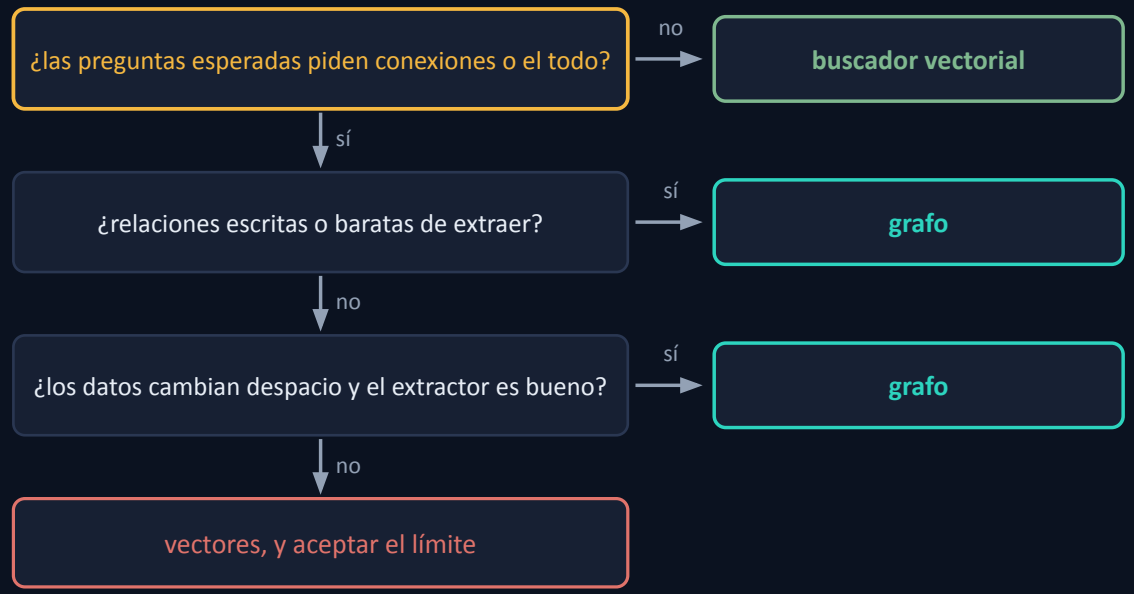
1. Preguntas de varios saltos.
2. Preguntas cuya respuesta es un conjunto.
3. Preguntas sobre los temas de un corpus.

Vale más si las relaciones están escritas o son baratas de extraer —las citas lo eran— y si los datos cambian despacio: el indexado se paga una vez.

No se arma si las preguntas esperadas son otras.

1. Datos sueltos, que se responden con un pasaje: vectores y re-puntuador ganan.
2. Si todo entra en la ventana, se pega.
3. Si los datos cambian todo el tiempo, el grafo se rehace todo el tiempo.
4. Con un extractor demasiado chico, sale con agujeros.

Con el sistema andando, se rutea. Cada pregunta que llega va al buscador que le sirve, grafo o vectores.



al diseñar: ¿grafo o no? · ya andando: rutear cada pregunta

Un grafo vale cuando las preguntas esperadas son sobre conexiones o sobre el todo; con el sistema andando, cada pregunta va al buscador que le sirve.

GRAFOS DE CONOCIMIENTO · SECCIÓN 5 DE 5

La memoria como grafo temporal

Aristas con dos relojes: la memoria del usuario y el corpus unidos por dos puentes, y lo que cuesta.

- Del parecido a la conexión: grafos de conocimiento
- Construcción del grafo
- Recuperación sobre grafos
- Consultas globales y comunidades
- **La memoria como grafo temporal**

La memoria también tiene conexiones

Volvemos a la memoria del usuario. En la memoria de largo plazo, cada hecho era una fila con su vector: “trabaja en una pesquera de Gdansk”, “consultó el reglamento del Báltico de 2013”. Son islas: ninguna sabe de la otra.

La pregunta que quedó abierta antes del recreo pide la cadena. “¿Qué reglamentos de los que consulté rigen donde trabajo ahora?” necesita el empleador vigente —que cambió—, el lugar, el mar, las normas de ese mar —en el grafo del corpus— y, de ésas, las que el usuario consultó —en los episodios—. Una memoria vectorial trae los registros que se parecen a la pregunta, no la cadena.

Y uno de los eslabones es de tiempo. Cuál empleador vale hoy no es una cuestión de parecido: “trabaja en Vigo” y “trabaja en Gdansk” se parecen igual a la pregunta.



los registros se parecen a la pregunta; la respuesta es un camino

Una memoria en filas sueltas no sabe que sus hechos están conectados, ni cuál vale hoy.

Un grafo de conocimiento temporal

Zep es un sistema de memoria para asistentes de IA. Es de una empresa del mismo nombre, así que sus cifras son de proveedor. Guarda la memoria como un grafo, y su motor, Graphiti, es abierto.

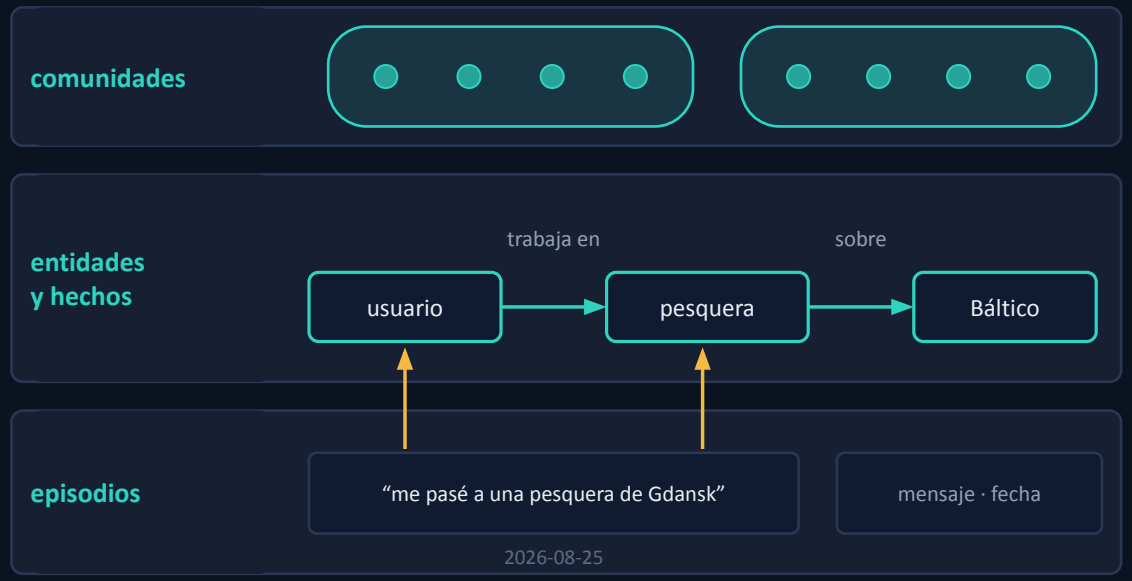
Tiene tres niveles.

1. Abajo, los episodios: los mensajes tal como se dijeron, con su fecha. Es la memoria episódica.
2. En el medio, las entidades y los hechos entre ellas: usuario, pesquera, Báltico, “trabaja en”. Es la memoria semántica, ahora con aristas.
3. Arriba, las comunidades de entidades relacionadas, como las del corpus.

Cada episodio pasa por lo que ya vimos. De “me pasé a una pesquera de Gdansk” se extrae el hecho usuario → trabaja en → pesquera de Gdansk, como se extrajo el grafo del corpus, y cada entidad se compara con las que ya existen: es la consolidación de la memoria, sobre un grafo.

Se lee de tres formas a la vez.

1. Por significado, con embeddings.
2. Por palabras, con BM25.
3. Por estructura, recorriendo el grafo desde las entidades de la pregunta.



cada episodio → los hechos que se extrajeron de él

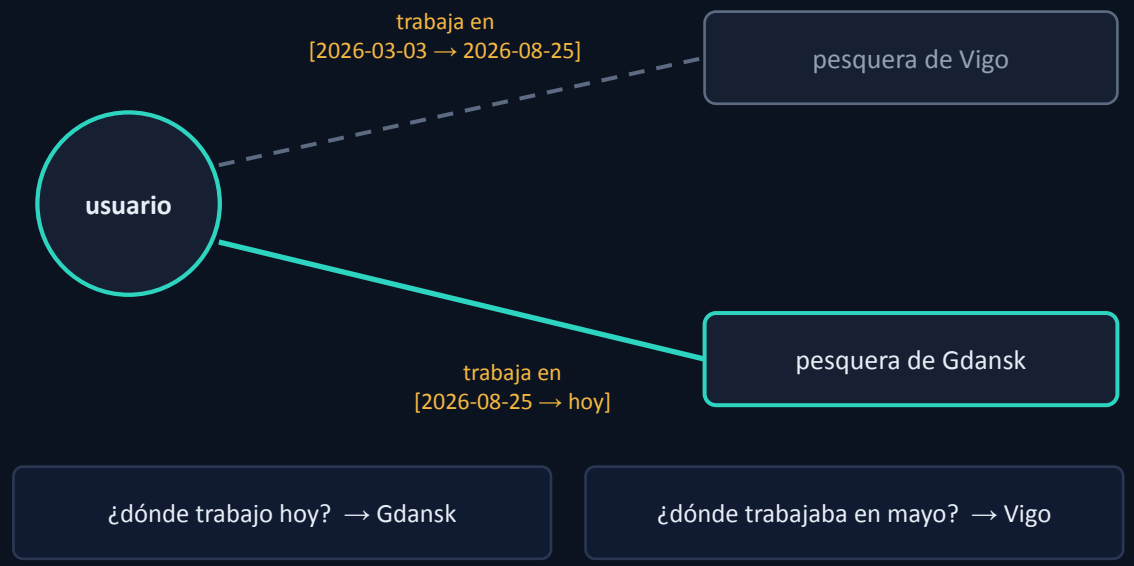
Un grafo temporal guarda tres cosas: lo que se dijo, los hechos que se sacaron y los temas que los agrupan.

Aristas con dos relojes

Cada arista lleva dos fechas. Una dice cuándo fue cierto el hecho en el mundo, desde y hasta cuándo. La otra, cuándo lo registró el sistema. Ya las vimos en la memoria sin grafo; en Graphiti viven en cada arista, y por eso se las llama aristas bitemporales (bi-temporal edges).

Una contradicción cierra la arista. Cuando llega “me pasé a una pesquera de Gdansk”, la arista “trabaja en ⟶ Vigo” recibe su fecha de fin y la nueva arranca ahí. No se borra nada: el grafo sigue sabiendo dónde trabajaba el usuario en mayo.

Recorrer en una fecha. Para “¿dónde trabajo?”, el recorrido sigue sólo las aristas vigentes hoy; para “¿dónde trabajaba en mayo?”, las vigentes en mayo.



no se borra nada: se recorre en una fecha

En un grafo temporal, cada arista sabe desde cuándo y hasta cuándo fue cierta. ▸ Notebook §19 — invalidar una arista y recorrer en una fecha.

La cadena, recorrida

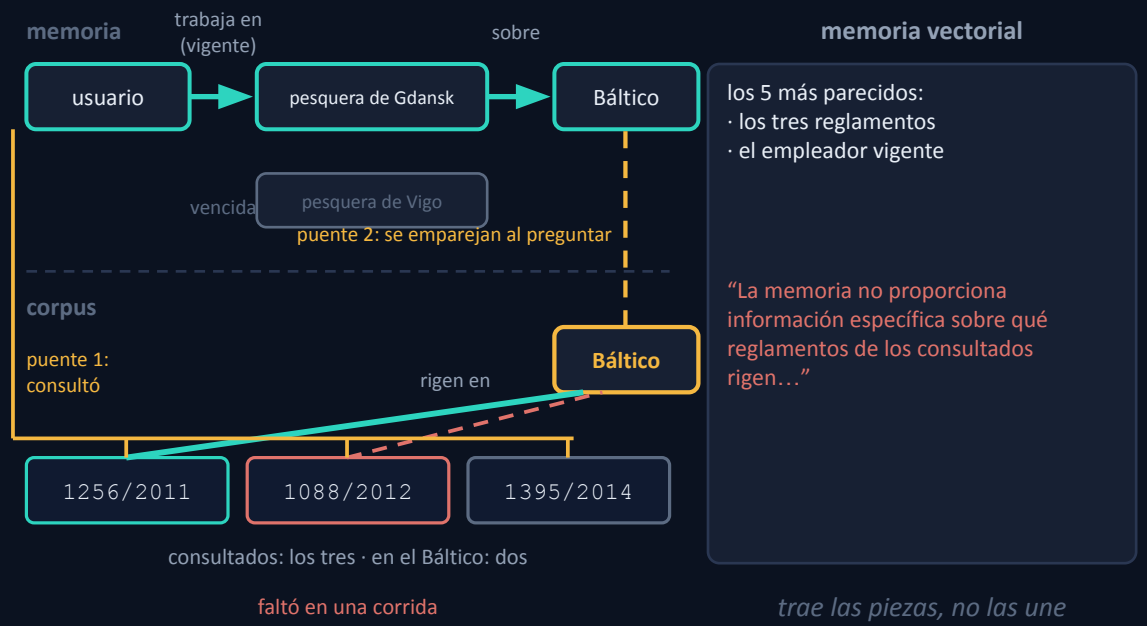
Dos grafos, unidos por dos puentes. El de la memoria del usuario y el de las normas. Primer puente: cada pregunta que hizo el usuario queda como una arista “consultó” hacia el reglamento que se trajo, que es un nodo del corpus. Segundo puente: al preguntar, el lugar de la memoria (“pesquera de Gdansk, sobre el Báltico”) se empareja con la zona del corpus (“Mar Báltico”) por parecido.

Con los puentes, la cadena es un camino.

1. usuario ⟶ trabaja en (vigente) ⟶ pesquera de Gdansk ⟶ sobre el Báltico.
2. Báltico ⟵ rigen en ⟵ reglamentos del corpus.
3. De esos reglamentos, los que el usuario consultó.

Cada eslabón trae su fecha o su pasaje: el camino es la respuesta y su explicación.

Una memoria sin grafo trae las piezas, no las une. Trae los tres reglamentos consultados y el empleador vigente, y el modelo no los une. El camino vale lo que valen sus eslabones: en una corrida faltó, en el grafo del corpus, la arista que ubica el reglamento de 2013 en el Báltico, y la respuesta salió con uno solo de los dos.



dos grafos, dos puentes: un camino

Memoria y corpus siguen siendo dos grafos, unidos por dos puentes: con ellos, la pregunta de varios saltos es un camino. ▸ Notebook §19 — la pregunta de la cadena, con memoria en grafo y con memoria vectorial.

Cuánto paga un grafo de memoria

Las ganancias publicadas son modestas. Mejoran poco a una memoria sin grafo, y las cifras las publica quien vende el sistema.

Cuesta en cada mensaje. Cada mensaje que entra pasa por extraer, resolver entidades y cerrar aristas viejas: varias llamadas al modelo por turno. Con modelos chicos falla más.

Conviene según la historia y las preguntas. Con pocas sesiones que entran en la ventana, volver a leer todo gana: si entra, se pega. Un grafo de memoria paga cuando la historia es larga y las preguntas son de conexiones y de tiempo.

Queda una pregunta abierta. Quién decide cuándo recorrer el grafo.



ganancias publicadas: modestas, y de los propios proveedores

con pocas sesiones que entran en la ventana, volver a leer gana

paga con historia larga y preguntas de conexiones y de tiempo

Un grafo de memoria compra conexiones y tiempo; se paga en llamadas, y con un modelo chico, en errores.

CONTROL Y SÍNTESIS · SECCIÓN 1 DE 2

Control de la memoria

Hasta acá decidió el programa. También puede decidir el modelo, y ahí empieza un agente.

Control de la memoria

Síntesis

QUIÉN DECIDE · LA PREGUNTA

Hasta acá decidió el programa

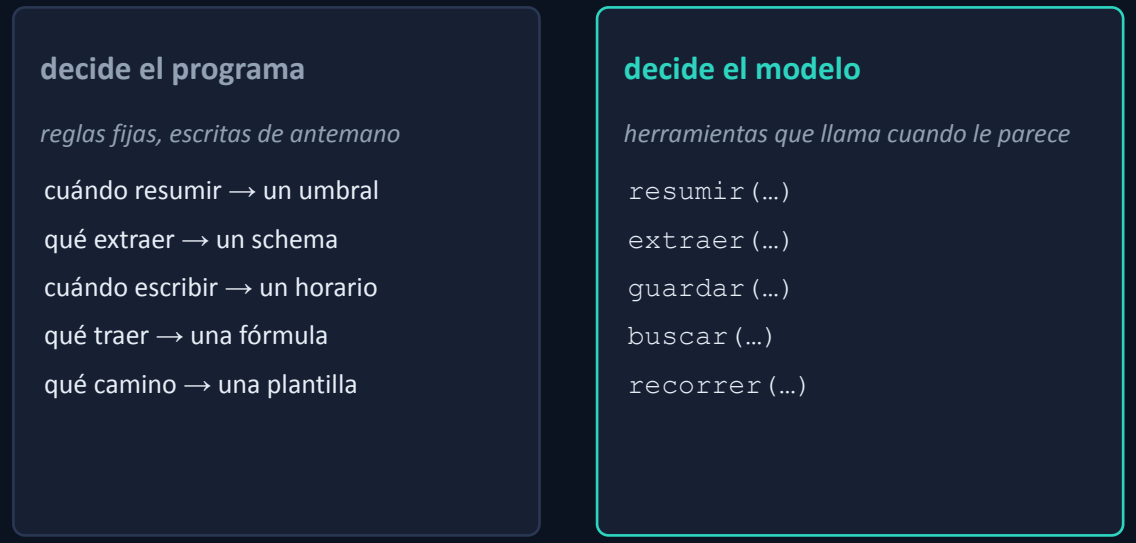
La pregunta madre tenía tres partes: qué vuelve a entrar, cuándo, y quién lo decide. Las dos primeras ocuparon la clase. Falta la última.

Hasta acá decidió nuestro código.

1. Cuándo resumir: un umbral.
2. Qué extraer: un schema.
3. Cuándo escribir: al final de la sesión, o en cada turno.
4. Qué traer: una suma de puntajes.
5. Qué relaciones existen: un schema.
6. Qué camino seguir: una plantilla.

Son reglas, con las virtudes de las reglas: predecibles, auditables, cuestan siempre lo mismo.

Y el defecto de las reglas: son ciegas a la tarea. El umbral resume cuando se llena la ventana, no cuando conviene. El schema extrae lo pensado de antemano, no lo que esta conversación necesita. El código no puede saber que este detalle, justo éste, va a importar. La alternativa es que lo decida el modelo, que sí está leyendo la conversación.



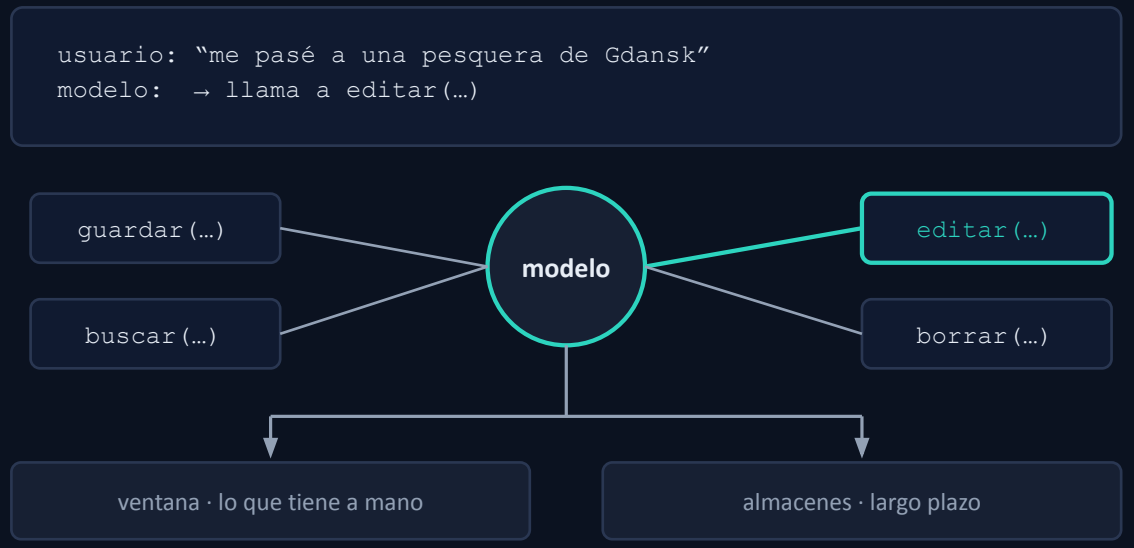
Hasta acá decidió el programa, con reglas predecibles y ciegas a la tarea. También puede decidir el modelo.

La memoria como herramienta

Darle al modelo herramientas para su propia memoria. Guardar, buscar, editar, borrar: funciones que el modelo llama a mitad de la conversación, con el function calling de prompt engineering. Cuando el usuario dice “me pasé a una pesquera de Gdansk”, es el modelo el que decide actualizar el registro.

Así funcionan los sistemas que más lo empujaron. En MemGPT, el modelo pagina su propio contexto entre niveles —lo que tiene a mano, la base de mensajes, el archivo de largo plazo—, y el aviso de memoria bajo presión, el del 70%, es su señal para guardar lo que le importe antes del desalojo. En Letta, su sucesor, el agente edita bloques de memoria con nombre y tamaño. Y en los productos: la memoria de ChatGPT, y la herramienta de memoria de la API de Anthropic, cuyas instrucciones empiezan con “ASSUME INTERRUPTION”: revisar la memoria antes de arrancar, porque la ventana se puede reiniciar en cualquier momento.

El mapa, con otro sistema operativo. Al principio de la clase, el programa hacía de sistema operativo. Ahora lo hace el modelo, con sus límites: depende de su criterio —un modelo chico guarda trivialidades u olvida guardar—, cada operación es otra llamada, y un modelo que decide cuándo usar herramientas ya es un agente.



el sistema operativo ahora es el modelo

MemGPT · Letta · la memoria de ChatGPT · la herramienta de memoria de Anthropic

Con herramientas para su propia memoria, el modelo pasa a ser su propio sistema operativo. Y un modelo que decide cuándo usar herramientas ya es un agente: la próxima unidad. ▸ Notebook §20 — el cambio de trabajo, con herramientas.

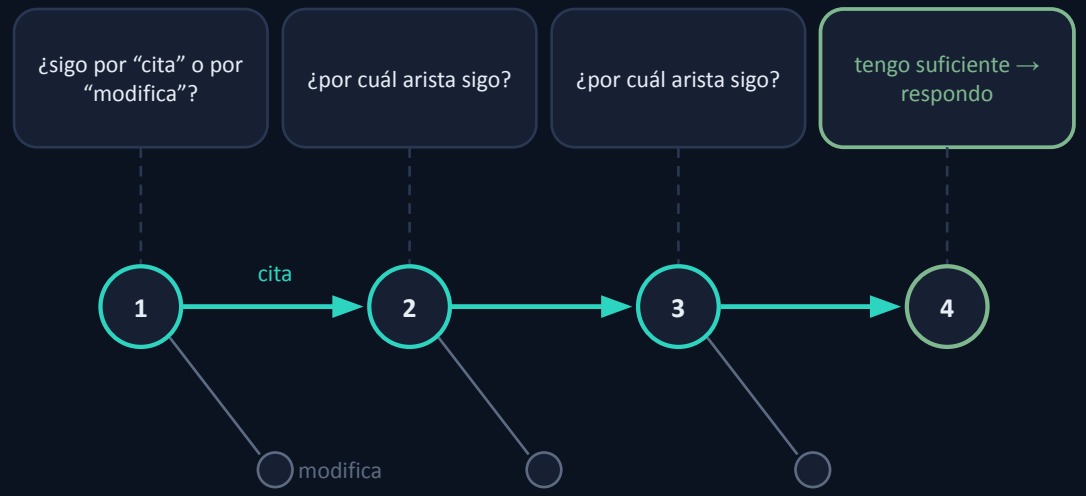
Un agente que recorre el grafo

Recorrer el grafo también puede decidirlo el modelo. Hasta acá, el camino lo fijaba el código: una plantilla, un vecindario de dos saltos, PageRank desde las entidades de la pregunta. La alternativa es que el modelo mire dónde está, elija la próxima arista y decida cuándo tiene suficiente.

Tres formas de hacerlo.

1. En Think-on-Graph, el modelo hace una búsqueda en haz sobre el grafo: en cada paso evalúa los caminos posibles y se queda con los mejores, sin ningún entrenamiento.
2. En SPINACH, de Stanford, un agente con herramientas para buscar entidades y ejecutar consultas resuelve la ambigüedad de los nombres como un paso más.
3. En Graph-CoT, el modelo alterna razonar, interactuar con el grafo y ejecutar, en un loop.

El mismo precio que la memoria como herramienta. Cada paso es otra llamada, el resultado depende del criterio del modelo, y un modelo que decide qué herramienta usar y cuándo parar ya es un agente: la próxima unidad.



el camino se arma de a un paso: cada paso, una llamada

Recorrer el grafo también puede decidirlo el modelo, paso a paso: eso ya es un agente.

Notas para uno mismo

La memoria de trabajo de un modelo es lo que escribió. Lo vimos con los tokens como cómputo: no hay un borrador oculto, el modelo razona sobre lo que ya escribió. Un trabajo de 2021 lo llamó scratchpad: dejar escritos los pasos intermedios mejora los problemas de varios pasos.

En las tareas largas, el scratchpad sale de la ventana. Un agente que trabaja durante horas escribe notas afuera y las relee. Anthropic describe sus agentes de larga duración como "engineers working in shifts" sin "memory of what happened on the previous shift": la solución es un archivo de progreso y el historial de commits, que cada turno nuevo lee primero. Y un truco que conecta con la curva en U: un equipo de producción hace que su agente reescriba la lista de pendientes al final del contexto en cada paso, para que el objetivo no se pierda en el medio.

Es memoria escrita por el modelo, para el modelo. Episódica, en cierto sentido, pero de una tarea, no de un usuario.



memoria de una tarea, no de un usuario

En una tarea larga, el modelo recuerda lo que escribe para sí: notas, un archivo de progreso, una lista de pendientes que vuelve a leer.

Aislar: una ventana por tarea

La cuarta flecha del mapa. En vez de una ventana que carga todo, se parte el trabajo: cada subtarea la hace un sub-agente con su propia ventana limpia, y devuelve sólo un resultado condensado al que coordina. Es aislar (isolate).

Comprime por diseño, y cuesta. En el sistema de investigación multi-agente de Anthropic, “subagents facilitate compression”: cada uno explora mucho y devuelve poco. Pero el sistema completo usa unas 15 veces los tokens de un chat.

Y tiene un contrapunto serio. Cognition, que construye agentes de código, advirtió que los agentes aislados toman decisiones que chocan entre sí, porque ninguno ve lo que decidió el otro: “share full agent traces”. En 2026 encontró una forma que funciona: que las escrituras queden en un solo hilo. Cómo coordinar varias ventanas es la unidad de sistemas multi-agente.



Aislar es la cuarta operación: varias ventanas chicas en vez de una enorme. Cuesta tokens y coordinación, y es la puerta a los sistemas multi-agente.

La memoria adentro de los pesos

Hoy toda la memoria vivió afuera del modelo. Porque los pesos están congelados: lo que el modelo vive en una conversación no cambia lo que es. Las personas consolidan lo que vivieron —en buena parte, mientras duermen—; los modelos de lenguaje no tienen nada equivalente. Ese aprendizaje continuo (continual learning) es, hoy, una ausencia.

La frontera intenta llevar la memoria adentro. Titans, presentado en NeurIPS 2025, agrega un módulo de memoria neural que aprende durante el uso; en sus palabras, la atención "performs as a short-term memory, while neural memory… acts as a long-term, more persistent, memory", y escala a más de 2 millones de tokens. DeepSeek propuso en 2026 memorias de consulta construidas adentro del modelo. Ninguna está en el stack de hoy.

Y una lectura distinta de lo que hicimos. Letta lo llama "continual learning in token space": aprender actualizando el contexto, no los pesos. Es exactamente lo que construyó esta clase.



toda la memoria de hoy circuló entre la ventana y los almacenes

Hoy toda la memoria vive afuera del modelo porque los pesos están congelados. La frontera es llevarla adentro: que el modelo aprenda de lo que vive.

CONTROL Y SÍNTESIS · SECCIÓN 2 DE 2

Síntesis

El mapa cobrado, la pregunta madre respondida, lo que medimos y el hilo con el resto del curso.

Control de la memoria
Síntesis

El mismo sistema, en un framework

Todo lo de hoy tiene nombre en los frameworks.

1. La lista de mensajes es el checkpointer de LangGraph: el estado de cada conversación se guarda por hilo, “so the thread can be resumed at any time”.
2. Recortar y resumir son operaciones de LangChain —recortar, borrar y resumir mensajes—, más un middleware de resumen que se dispara a N tokens y conserva los últimos M mensajes.
3. El almacén de largo plazo es el store de LangGraph, compartido entre hilos y separado por usuario.
4. Los tipos de memoria son los bloques de memoria de LlamaIndex: uno estático (el perfil), uno de extracción de hechos (la semántica) y uno vectorial (la episódica), a los que se vuelca la conversación al pasar el 70% de su presupuesto.

Por eso se construyó a mano primero. Como con RAG: para saber qué está configurando cada framework.

La pregunta antes de adoptar uno: qué decide por su cuenta. Cuándo resume, qué guarda, qué presupuesto de tokens supone, a qué modelo llama. El middleware de resumen de LangChain, por ejemplo, cuenta los tokens con una aproximación y no con el tokenizer del modelo, resume con instrucciones en inglés y le devuelve el resumen al modelo como un mensaje del usuario. Cada respuesta es una de las decisiones de esta clase, tomada por otro.



¿cuándo resume, qué guarda, con qué presupuesto?

Laboratorio: los 30.000 tokens de memoria por defecto de LlamaIndex superan la ventana que Ollama da por defecto en una T4: hay que alinearlos. Los docs usan modelos pagos.

Lo que se construyó a mano tiene nombre en el framework. La pregunta al adoptarlo es qué decide por su cuenta: cuándo resume, qué guarda y con qué presupuesto. ▸ Notebook §21 — el mismo sistema, en LangGraph.

El grafo, en un framework

Lo de la tercera parte también tiene nombre propio.

1. La librería de grafos de Neo4j trae un pipeline que extrae contra un schema y una colección de recuperadores, de la búsqueda local a Text2Cypher.
2. El índice de grafo de propiedades de LlamaIndex trae los tres extractores, del libre al estricto.
3. LangChain convierte documentos en grafo con su LLMGraphTransformer.
4. Graphiti es la memoria temporal, con un servidor MCP.
5. Nano-graphrag reimplementa GraphRAG en unas mil cien líneas que se pueden leer; el GraphRAG de Microsoft está hoy en mantenimiento.

La misma pregunta: qué decide cada uno por su cuenta. El schema, si no se le da uno. Cómo resuelve las entidades. Si deja que el modelo escriba la consulta, con lo que eso implica para la seguridad. Y con qué modelo se probó: GraphRAG se armó y se probó con modelos de OpenAI —"built and tested using OpenAI models"— y avisa de JSON mal formado con servidores como Ollama.

Cada respuesta es una decisión de esta parte, tomada por otro. Como con la memoria, construirlo a mano primero es lo que permite leer qué está configurando el framework.

a mano	dónde vive en cada framework
el grafo de citas	una base de grafos · Neo4j
la extracción con schema	pipeline de Neo4j · LlamaIndex · LLMGraphTransformer
la resolución de entidades	la decide cada framework
la búsqueda local	recuperadores de Neo4j · Text2Cypher
las comunidades	GraphRAG · nano-graphrag
la memoria temporal	Graphiti · servidor MCP

Laboratorio: en producción el grafo vive en una base como Neo4j y se consulta con Cypher; el notebook usó NetworkX, que corre en Colab sin servidor ni cuenta.

Un framework de grafos decide el schema, la resolución de entidades y quién escribe la consulta: son las decisiones de esta parte, tomadas por otro.

Cuatro operaciones

El mapa de la apertura, cobrado. Cada técnica de hoy es una de cuatro operaciones sobre la ventana: las cuatro flechas que dibujamos al principio.

Las cuatro, llenas.

1. Escribir afuera (write): descargar con un puntero, la memoria de largo plazo, las notas del scratchpad, y el grafo —el de dominio, extraído del corpus, y el de memoria temporal—.
2. Seleccionar qué entra (select): la pregunta reescrita, la carga justo a tiempo, traer memorias por relevancia, recencia e importancia, y recorrer el grafo —la búsqueda local, el camino de varios saltos, PageRank personalizado—.
3. Comprimir lo que está adentro (compress): recortar, resumir, las cadenas de resumen, comprimir pasajes y los resúmenes de comunidades.
4. Aislar (isolate): los sub-agentes con su propia ventana.

Lo que en prompt engineering era una lista de tácticas. Ahora cada una tiene su mecanismo, su costo y su forma de fallar.



cuatro flechas, dos semanas de técnicas

Todas las técnicas de hoy son cuatro operaciones sobre la ventana: escribir afuera, traer, comprimir y aislar.

CIERRE · LA SÍNTESIS

Lo que no está en la ventana no existe

El modelo no cambió en toda la clase. Es el mismo de prompt engineering y de RAG, con los mismos pesos. Lo que cambió es qué entra a su ventana, cuándo y quién lo decide.

La pregunta madre, respondida.

1. Decidir al escribir —resumir, extraer— sale barato después, pero es ciego a la pregunta que va a venir.
2. Decidir al leer —guardar todo y buscar cuando llega la pregunta— se paga en cada pregunta y depende de que la búsqueda acierte.

Los sistemas reales hacen las dos cosas: un perfil que se precarga y una colección que se busca; un resumen arriba y lo descargado afuera. Y un grafo lleva la elección al extremo: GraphRAG resume al escribir, LazyGraphRAG espera la pregunta.

Y el contraste que ordenó la segunda mitad. RAG leía un corpus que escribió otro. La memoria escribe el suyo, con el mismo modelo que conversa, y por eso sus errores persisten. Lo que se recupera sigue siendo el techo de la respuesta; ahora, además, el sistema escribe lo que después va a recuperar, y puede escribirlo como vínculos: la similitud trae lo que se parece; un grafo trae lo que está conectado.

momento	lo que se rompió	lo que lo arregló
turno 2	la pregunta depende de la anterior	reescribir la pregunta
turno 20	mandar todo cuesta, y la ventana se acaba	cache · contar · presupuesto
turno 200	la conversación ya no entra	recortar · resumir · descargar
día siguiente	la sesión anterior no existe	memoria de largo plazo
con grafos	la similitud no recorre cadenas	el grafo

el mismo modelo en las cinco filas: cambió qué entra a la ventana

El modelo no recuerda: lee. Todo lo que llamamos memoria es decidir qué vuelve a entrar a la ventana, cuándo se decide y quién lo decide.

Qué mide, y qué no mide, lo que medimos

Lo que medimos. Cada mecanismo una vez, en un caso: la consulta reescrita contra el turno solo, el hecho plantado bajo cada política de olvido, dos preguntas por habilidad de memoria; y en el grafo, las preguntas de conexión contra los vectores, la auditoría de la extracción y lo que cubre cada respuesta a la pregunta global. Es ilustrativo, no estadístico: una conversación no es una muestra.

Lo que aprendimos sobre medir.

1. Que los benchmarks miden lo que su puntuación cuenta, como el de la frase en inglés.
2. Que el régimen decide quién gana: donde el historial entra, re-leer todo es difícil de superar.
3. Que los proveedores se miden a sí mismos.

Lo que no medimos. Los efectos de largo plazo —filtración, refuerzo de sesgos—, que aparecen a lo largo de muchas sesiones. El costo a escala, con miles de usuarios. El aislamiento entre usuarios. Y, en el grafo, la extracción con un modelo grande, el corpus entero y la evaluación con juez LLM que usan los papers. Saber si una memoria sirve en un sistema real pide medirla por habilidad, en su régimen, contra las dos vallas: es el trabajo del segundo proyecto.

medimos	aprendimos	no medimos
consulta reescrita vs turno solo	el benchmark mide lo que su puntuación cuenta	largo plazo: filtración, refuerzo de sesgos
hecho plantado bajo cada política de olvido	el régimen decide quién gana	costo y aislamiento, a escala
grafo vs vectores · la pregunta global	los proveedores se miden a sí mismos	grafo: modelo grande, corpus entero, juez LLM

ilustrativo, no estadístico: una conversación no es una muestra

Medimos cada mecanismo una vez, en un caso. Saber si una memoria sirve pide medirla por habilidad, en su régimen y contra re-leer todo.

El hilo con el resto del curso

Hacia atrás.

1. El bigrama no tenía memoria: veía un carácter.
2. La atención fue “darle memoria al modelo”, pero sólo adentro de la ventana.
3. La tokenización fijó la unidad del presupuesto y los tokens especiales de la plantilla de chat.
4. Los embeddings ahora traen memorias.
5. La complacencia que vimos en alineamiento se amplifica con la memoria.
6. Prompt engineering convirtió el contexto en un recurso finito y dio los schemas para reescribir y extraer.
7. RAG leía un corpus; hoy el sistema escribe uno.
8. Los grafos del curso —el de cómputo, el de la atención, HNSW— dejaron lugar a uno cuyas aristas son hechos.

Hacia adelante.

1. La unidad de agentes: la memoria como herramienta y el grafo recorrido paso a paso, en un loop que decide cuándo usarlos.
2. La clase de planificación: reflexionar sobre los episodios para actuar mejor.
3. Los sistemas multi-agente: aislar ventanas, compartir memoria y defenderla de inyecciones, con LangGraph, cuyo grafo es de control y no de conocimiento.

Tres veces le dimos memoria al modelo.

1. La atención, adentro de la ventana.
2. RAG, con un corpus ajeno.
3. Hoy, con un corpus propio.



El curso le dio memoria al modelo tres veces: la atención adentro de la ventana, RAG con un corpus ajeno y hoy un corpus propio. Lo que sigue es que decida él.

Cierre y recursos

Lo que exhiben los notebooks. El primero corre la conversación de punta a punta sobre el corpus de la clase de RAG: el modelo sin estado y la plantilla de chat; la pregunta que depende de la anterior, buscada sola, con toda la conversación y reescrita; el costo de mandar todo y el cache de prefijos; el desborde silencioso, provocado a propósito; el ensamblador con su presupuesto; las políticas de olvido sobre un hecho plantado; una cadena de resumen; la memoria de largo plazo con sus cuatro operaciones y sus dos relojes; la vara por habilidad; y el mismo sistema en LangGraph. El segundo arma el grafo de citas del corpus, extrae entidades y relaciones de los actos de pesca, recorre el grafo contra los vectores, detecta comunidades para la pregunta global y une la memoria del usuario con el corpus en un mismo grafo. En clase se ven salidas capturadas.

Lo que sigue. La unidad de agentes: un modelo que decide cuándo buscar, cuándo recordar y por dónde recorrer.

Núcleo. Packer et al. 2023 (MemGPT: Towards LLMs as Operating Systems) · Park et al. 2023 (Generative Agents) · Sumers et al. 2024 (Cognitive Architectures for Language Agents, CoALA) · Wu et al. 2025 (LongMemEval) · Peng et al. 2025 (Graph Retrieval-Augmented Generation: A Survey)

Para afianzar. Liu et al. 2024 (Lost in the Middle) · Hsieh et al. 2024 (RULER) · Xiao et al. 2024 (StreamingLLM) · Kwon et al. 2023 (PagedAttention) · Wang et al. 2025 (Recursively Summarizing Enables Long-Term Dialogue Memory) · Chhikara et al. 2025 (Mem0) · Rasmussen et al. 2025 (Zep) · Sarthi et al. 2024 (RAPTOR) · Edge et al. 2024 (GraphRAG) · Gutiérrez et al. 2024 (HippoRAG) · Xiang et al. 2026 (When to use Graphs in RAG) · Zhang et al. 2026 (ACE) · Pulipaka et al. 2026 (PersistBench) · Katsis et al. 2025 (MTRAG) · Ye et al. 2023 (reescritura informativa de consultas)

Práctica. Anthropic, Effective context engineering for AI agents · Lance Martin, Context Engineering for Agents · Manus, Context Engineering for AI Agents · docs de memoria de LangGraph · DeepLearning.AI (LLMs as Operating Systems: Agent Memory; Long-Term Agentic Memory with LangGraph; Knowledge Graphs for RAG) · Neo4j, GraphRAG Pattern Catalog · docs de GraphRAG de Microsoft

Varios hilos salen de las charlas de Andrej Karpathy y su nota “llm-wiki”: la amnesia de Memento, los pesos como recuerdo difuso, la ventana como memoria de trabajo, el LLM como sistema operativo, el saber que se compila una vez. Un modelo que decide qué recordar ya es un agente. ▸ Agentes.